Integrating symbolic execution with LLMs for automated generation of C program specifications

Fanpeng Yang, Xu Ma, Shuling Wang*, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu*

***Abstract*—Automatically generating formal specifications including loop invariants, preconditions, and postconditions for legacy code is critical for program understanding, reuse, and verification. However, the inherent complexity of control and data structures in programs makes this task particularly challenging, especially when no verification goal is provided to drive synthesis. This paper presents SESPEC, a framework that integrates symbolic execution with large language models (LLMs) and a formal verifier (Frama-C/WP) to synthesize verifier-checked C program specifications. For loop-free fragments fully covered by the symbolic executor, SESPEC computes exact path-sensitive postconditions, composing the specifications of ordinary (non-recursive) callees along the way; at the remaining boundaries (loops, recursive data structures, and fragments outside the symbolic executor), it uses the available symbolic facts to guide verifier-checked LLM synthesis. During symbolic execution, SESPEC decomposes the synthesis task by pausing at program boundaries, where the symbolic state is injected into annotation templates and LLM prompts. When a specification is faulty, verifier feedback is combined with symbolic information to refine the failed candidate rather than re-sampling blindly. This design supports goal-independent specification generation, because it derives functional relations from symbolic states and verifier feedback rather than relying solely on externally provided assertions. Through extensive evaluation across numerical, pointer, struct, and linked-list benchmark families and an additional goal-independent benchmark (SESPEC-400), SESPEC outperforms the state-of-the-art LLM-based tools, including in the goal-independent setting where no verification target is provided.**

***Index Terms*—Program specification generation, loop invariants, symbolic execution, large language models, formal verification.**

I. INTRODUCTION

Across software domains, from general-purpose systems to long-lived industrial control codebases, a vast body of legacy code lacks formal specifications, i.e., precise, verifiable descriptions of program behavior such as preconditions, postconditions, and loop invariants. Automating specification generation is therefore essential for program comprehension, reuse, and verification. The task is challenging due to the interplay of complex control structures (nested branches, loops, function calls) and data representations (structs, pointers, inductive datatypes). In this work, we tackle *goal-independent* functional specification generation: inferring verifier-checked preconditions, postconditions, and loop invariants that capture functional behavior without requiring a user-supplied assertion as the synthesis target. The goal is not to prove one preselected property, but to recover verifier-checked specifications that capture a program's functional behavior, whether or not symbolic execution can derive them exactly.

Existing work on program specification generation has largely focused on loop invariants. Traditional symbolic methods such as constraint solving [1], [2], Craig interpolation [3]–[5], recurrence analysis [6], [7], shape analysis [8], and dynamic methods such as Daikon [9] are each restricted to particular invariant forms or program classes; recent ML-augmented or SMT-augmented methods [10]–[12] are similarly confined to numeric programs. LLM-based approaches have begun to broaden the program coverage, but tend to produce specifications that are either ill-formed, too weak, or both, and recent integrations with static/formal techniques [13]–[16] only partially address this. AUTOSPEC [13] and Preguss [16] are goal-dependent (requiring a predefined verification target or RTE target) and degrade on complex data structures; SPECGEN [15] is goal-independent but only in the Java/JML setting, and "verifier-accepted" alone says nothing about strength. These limitations leave open the problem of generating goal-independent, semantically informative, verifier-checked specifications for C programs with loops, calls, pointers, structs, and inductive data.

To address this open problem, we propose SESPEC, a framework that integrates symbolic execution, LLM-based synthesis, and formal verification. The key idea is to let symbolic execution drive the process. The analysis proceeds through the program and pauses at program boundaries (loops, calls, and recursive constructs), turning one whole-function specification task into a sequence of smaller local synthesis problems. On loop-free segments fully handled by the flattened-memory symbolic executor, symbolic execution computes an exact path-sensitive postcondition, which is the strongest postcondition within that model; outside this envelope, symbolic execution provides partial semantic anchors for LLM synthesis. At each boundary, the symbolic state, i.e. the symbolic store of variable values, the path conditions, and the types of the in-scope variables, collectively the *local symbolic context*, is injected into invariant templates and LLM prompts. When a candidate fails, formal verification feedback is combined with symbolic information to refine the failed candidate rather than

*Corresponding authors: Shuling Wang (e-mail: wangsl@ios.ac.cn), Bin Gu (e-mail: gubin@ios.ac.cn).

F. Yang and X. Ma are with the Key Laboratory of System Software, Institute of Software, Chinese Academy of Sciences and also with the University of Chinese Academy of Sciences, Beijing, China (e-mail: yangfp@ios.ac.cn; maxu@ios.ac.cn)

S. Wang and X. Xu are with the National Key Laboratory of Space Integrated Information Systems, Institute of Software, Chinese Academy of Sciences, Beijing, China (e-mail: wangsl@ios.ac.cn; xux@ios.ac.cn).

Q. Cao is with the School of Computer Science and School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China (e-mail: caoqinxiang@sjtu.edu.cn).

N. Zhan is with the School of Computer Science, Peking University, Beijing, China (e-mail: znj@ios.ac.cn).

X. Li and B. Gu are with the Beijing Institute of Control Engineering, Beijing, China (e-mail: 1351127502@126.com; gubin@ios.ac.cn).

re-sampling blindly.

We implement SESPEC on top of QCP's symbolic execution engine [17], [18] and the Frama-C/WP verifier [19]. We evaluate SESPEC on benchmark families from prior invariant- and specification-generation work, covering numerical loops, multi-/nested loops, pointers, structs, and singly linked lists, together with an ablation study of each component. On numerical benchmarks, SESPEC approaches tools specialized for numerical programs and consistently outperforms recent LLM-based approaches such as AUTOSPEC [13] and ACInv [14]; on the pointer-, struct-, and list-heavy families that prior LLM-based tools handle poorly, it outperforms AUTOSPEC by a clear margin. On **SESPEC-400**, a 400-program benchmark with all assertions removed, SESPEC substantially outperforms single-shot LLM and AUTOSPEC baselines, and the specification comparison finds that its specifications provide more informative postconditions and stronger loop invariants. These results show that the symbolic-execution-guided design generalizes from the goal-dependent to the goal-independent setting without specialization.

In summary, the key contributions of this work are:

- **Symbolic-execution-guided specification synthesis**: A framework that drives LLM-based specification generation with path-sensitive symbolic states: exact semantic summaries on fully covered segments, and local symbolic context anchoring templates and prompts elsewhere.
- **Verifier-guided iterative refinement**: A multi-layer refinement mechanism that classifies Frama-C/WP failures into diagnostic categories and applies targeted repair strategies, instead of blind re-sampling.
- **Goal-independent evaluation**: **SESPEC-400**, a benchmark of 400 C programs with assertions removed, used to conduct what we believe is the first empirical study of goal-independent specification generation, covering validity, strength, failure modes, and per-program cost.

The rest of the paper is organized as follows. Section II gives a motivating example and an overview; Sections III–IV present the core techniques of SESPEC; Sections V–VI report the empirical studies; Section VII surveys related work, and Section VIII concludes. System design, case studies, limitations, and threats to validity appear in Appendix E, Appendix J, Appendix K, and Appendix L, respectively.

II. A MOTIVATING EXAMPLE AND OUR APPROACH

Fig. 1 shows a fragment adapted from an industrial navigation component. Its `CheckCal` struct contains an array `pkv[10]`, its valid length `len`, and a sum field `chksum`. The caller `foo` invokes `CheckCalFun`, which sums the valid elements into `chksum`; an assertion in `foo` states the verification goal. This example raises three challenges:

- **Loop reasoning:** The summation loop `for(; i<pIp->len; i++)` requires determining whether the initial state allows loop entry and, if it executes, establishing invariants that relate the partial sum to the processed array prefix.
- **Structs and pointers:** The procedures read and write struct fields through pointers, so specifications must account for aliasing and side effects: memory safety for dereferencing `pIp->pkv[i]`, the write effect on `pIp->chksum`, and frame properties for unchanged fields of `pIp`.
- **Inductive predicates:** Aggregate array properties such as sums are most naturally expressed with inductive predicates or logic functions; writing them in a verifier-friendly form and proving the resulting annotations remains a central challenge.

Fig. 1 annotates the example with ACSL `requires`, `ensures`, `loop invariant`, and `assert` clauses. SESPEC generates the default preconditions, postconditions, and loop invariants; the assertion goal and additional `CheckCalFun` preconditions are supplied by the designers.

Before presenting our method, we attempted to generate the specification for this example using GPT-4o, Claude 3.7 Sonnet, and AUTOSPEC, given both the preconditions and the verification goal. The loop invariants they produced, shown below, were all found to be incorrect:

GPT-4O
```
loop invariant chksum == \sum(0, i-1,
  \lambda integer k; pIp->pkv[k]);
```
CLAUDE 3.7 SONNET
```
loop invariant chksum ==
  \sum(integer k = 0; k < i; pIp->pkv[k]);
```
AUTOSPEC
```
loop invariant chksum == \sum(0, i-1, pIp->pkv);
```

All three candidates include `0 <= i <= pIp->len` and a partial-sum relation, but omit the zero-iteration case (`pIp->len <= 0`); even a well-formed version therefore fails the base case. Explicitly prompting GPT-4o about this case did not produce a conditional invariant. Moreover, all three use `\sum` without defining an ACSL logic function; their lambda, quantified-binder, and raw-pointer variants are rejected by Frama-C/WP.

A. Our Approach

Fig. 2 shows an overview of our approach. Given a function and an optional verification goal, the objective is to automatically generate loop invariants and function pre-/postconditions. If a verification goal is provided, these specifications should be sufficient to prove it; otherwise, they should capture useful functional relations exposed by the available symbolic states. Following the three design choices of Section I, symbolic execution drives the pipeline and computes path-sensitive symbolic information at the current boundary; SESPEC injects these facts into invariant templates and ACSL skeletons, while a lightweight static route selects supporting examples and ACSL guidance. The LLM therefore receives both semantic facts from the current symbolic state and prompt support material matched to the program's syntactic features. The verifier then checks the candidate, and if it fails, the parsed verifier feedback is combined with the same local symbolic context to drive iterative refinement until a verifier-checked summary is produced. Each generation call is issued at a symbolic-execution boundary — a loop, an unspecified callee, a recursive data structure, or a recursive function — and once a verifier-checked summary

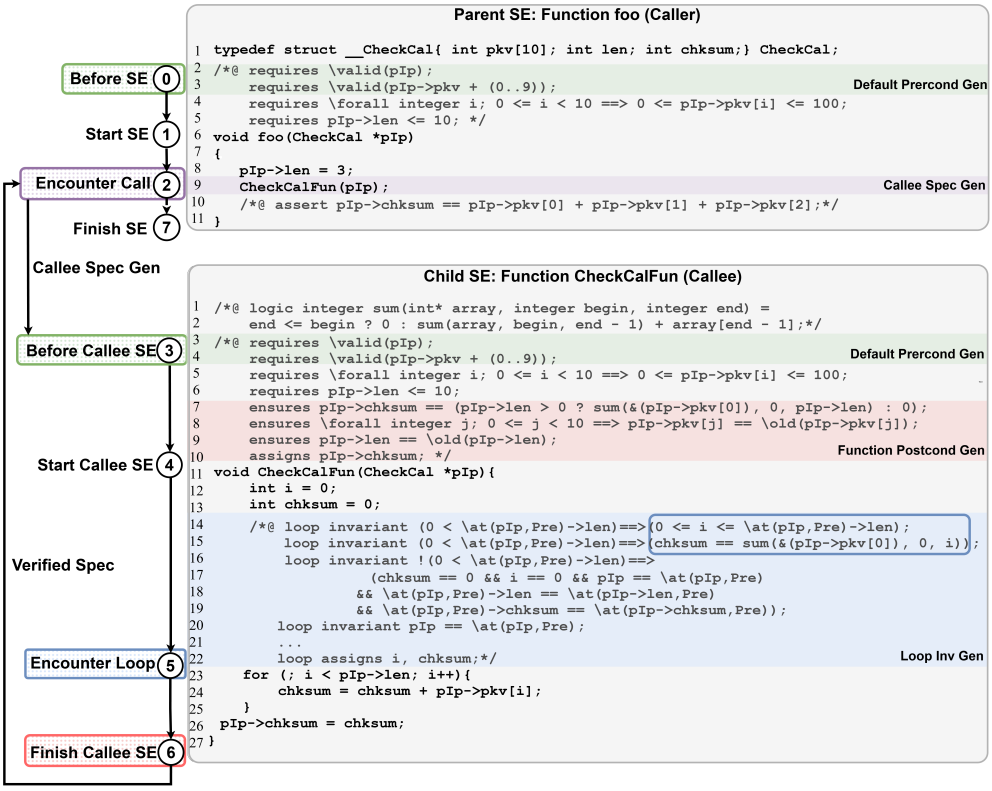


Fig. 1. Motivating Example

is obtained, symbolic execution resumes from the same point using that summary.

Fig. 2 divides function-specification generation into four symbolic-execution-driven stages and an LLM fallback (Section III-D) for self-recursion, recursive data structures, external calls, path explosion, floating-point arithmetic, and runtime errors. The first stage, **Default Precondition Generation**, creates memory-safety conditions for symbolic execution (node ⓪ in Fig. 1); Section III-B gives the details.

From the default precondition, symbolic execution starts at function entry (node ① in Fig. 1). At a call (node ②), SESPEC pauses to recursively generate the callee specification (nodes ③–⑥), then applies that contract at the call site and resumes the caller. A call stack handles nested calls; generated specifications are stored for reuse, yielding bottom-up processing.

At a loop (node ⑤ in Fig. 1), symbolic execution pauses for **Loop Invariant Generation**. Loop Analysis uses the entry state and body to identify unchanged variables (`pIp->len`, `pIp`, `pIp->pkv`) and inductive variables (`i`, `chksum`). It records the guard `i < pIp->len` and the condition for at least one iteration, `0 < \at(pIp, Pre)->len`. These facts instantiate ACSL templates with guards, frame clauses, and relation placeholders, illustrated below (the complete annotations appear in Fig. 1).

```
loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_i;
loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_chksum;
loop invariant pIp == \at(pIp, Pre);
```

These skeletons provide specific `PLACE_HOLDER` tags for the

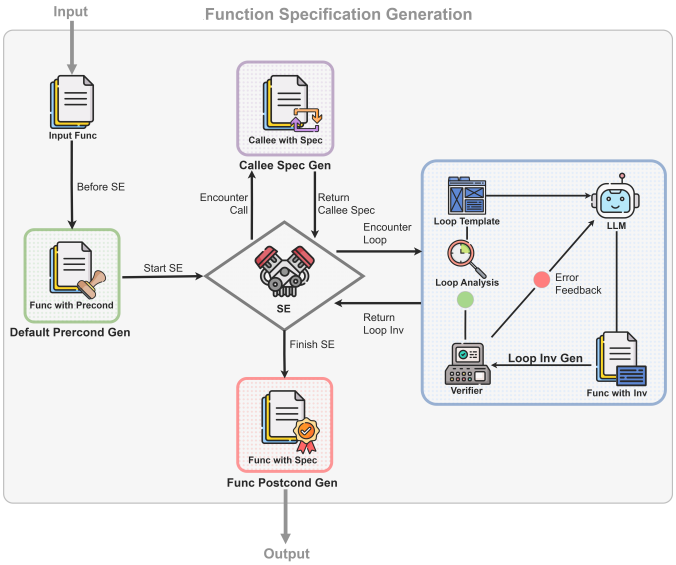


Fig. 2. Overview of Our Approach

LLM to populate (marked as the blue boxes in Fig. 1). Guided by these structured templates and symbolic execution results, the LLM produces loop invariants that capture useful relations among different types of variables. This symbolic-state-guided structure narrows the candidate space and makes key logical constraints explicit before the LLM fills the placeholders. For instance, the generated invariants correctly incorporate the loop-entry condition `(0 < \at(pIp, Pre)->len)`—the critical logical constraint whose omission caused the pure-LLM failures in Section II.

During Loop Invariant Generation, the verifier-guided refinement procedure is iteratively invoked to assess both the validity of the generated invariants and their usefulness for proving the given verification goal when such a goal is available. Depending on verifier feedback, different refinement strategies guide the LLM toward verifier-checked invariants.

When symbolic execution of the callee is completed (node ⑥ in Fig. 1), **Function Postcondition Generation** (marked in red in Fig. 2) is triggered. The resulting symbolic states are used to synthesize a postcondition that summarizes the callee's functional behavior. Upon completion, the generated pre-/postconditions are returned to the call site and reused as a functional summary in the caller (node ⑦ in Fig. 1), enabling symbolic execution of the caller to resume without re-analyzing the callee's internal control flow.

This example illustrates the full pipeline; goal-independent generation is evaluated separately in Section VI.

III. Pre-/Postcondition Generation

This section first introduces symbolic-execution-based specification generation for code segments without loops, and then presents the generation of general function specifications. Loop-invariant generation is deferred to the next section.

A. Symbolic execution and strongest postcondition

Before stating the proposition, we fix its assumptions.

At program point p, our *flattened memory model* starts from in-scope variables and parameters and enumerates a finite set $\mathcal{L}_p$ of atomic cells reached by pointer dereferences, struct-field selection, and element-wise traversal of statically known or parameter-bounded arrays. Each scalar or pointer cell receives a typed symbolic variable. Access paths resolved to the same cell share a location; pairs resolved as distinct are marked separated. Values, aliasing, and updates are therefore expressed over $\mathcal{L}_p$.

The model produces only finite, first-order facts over named cells; it does not use recursive or inductive predicates. These facts serve as preconditions and postconditions for two backends: QCP (in QCPL) for symbolic execution, and Frama-C/WP (in ACSL) for verification. What limits the expressiveness of the facts is Frama-C/WP's requirement for automatic SMT discharge: because the verification step must be fully automatic, the facts are expressed in first-order logic without recursive or inductive predicates. Hence the model is deliberately finite and handles only memory accesses whose target locations are statically enumerable. It supports scalars, structs, fixed arrays, and pointers with statically resolvable aliasing. Unbounded or recursive structures (e.g., linked lists and trees), dynamic unbounded allocation, and accesses with a symbolic offset (e.g., pointer arithmetic) fall outside the exact fragment and trigger the LLM fallback described in Section III-D.

Within this model, the symbolic execution of statements maintains a *symbolic state* $(PC_\pi, \sigma_{p,\pi})$ for each execution path π. At a program point p on path π, let $\mathcal{L}_p^\pi$ denote the set of symbolic locations currently tracked by the executor. This set includes the function-entry locations $\mathcal{L}_{p_0}$ reachable from the function parameters, together with the local variables that are in scope at p along π (hence $\mathcal{L}_{p_0} \subseteq \mathcal{L}_p^\pi$).

- PC_π denotes the path condition, i.e., a logical formula over function-entry symbolic values and constants that encodes the conditions needed to follow the path π.
- $\sigma_{p,\pi}$ denotes the symbolic store at program point p along path π, i.e., a mapping from each currently tracked symbolic location in $\mathcal{L}_p^\pi$ to a symbolic expression over the function-entry values. Thus, $\mathrm{dom}(\sigma_{p,\pi}) = \mathcal{L}_p^\pi$. We express the symbolic store $\sigma_{p,\pi}$ as the following logical relation:

$$\rho(\sigma_{p,\pi}) \;\equiv\; \bigwedge_{\ell \in \mathcal{L}_p^\pi} \ell^p = \sigma_{p,\pi}(\ell),$$

  where ℓ^p denotes the value of location ℓ at program point p, and $\sigma_{p,\pi}(\ell)$ is a symbolic expression over the function-entry values. Thus, $\rho(\sigma_{p,\pi})$ relates the function-entry state to the state at p over the tracked locations.

The symbolic execution of a program is defined as a transformation over sets of symbolic states, branching at conditional control flow points. Formally, execution begins at the function entry point p_0 with the initial symbolic state (PC_0, σ_0), where σ_0 assigns an initial symbolic value to each location in the entry point location set $\mathcal{L}_{p_0}$, and PC_0 is the precondition constraining those initial values. For a program point p, let the set of all symbolic states reachable from p_0 be $\{(PC_{\pi_i}, \sigma_{p,\pi_i})\}_{i=1}^m$, where π_i ranges over all paths from p_0 to p. The pairs of function-entry states and reached states at p, represented over the tracked locations, satisfy

$$\bigvee_{i=1}^{m} \left(PC_{\pi_i} \wedge \rho(\sigma_{p,\pi_i})\right).$$

The exact claim applies only to the finite set of tracked locations $\mathcal{L}_p^\pi$, and memory locations outside this set are not constrained by the resulting formula. The path condition PC_{π_i} is expressed over the same function-entry symbolic values and includes the starting precondition. At the entry point p_0, the store relation is the identity relation, so the corresponding formula is PC_0 together with those identity equalities.

Given a segment $\mathcal{F}$, precondition P, and start point p_{start}, $\mathrm{SEStart}(\mathcal{F}, P, p_{start})$ explores feasible paths to the next boundary p (a loop entry, call, or exit of $\mathcal{F}$) and returns p. The traversed segment contains no loop or call, so its input/output behavior can be summarized exactly.

We first define the notion of a *strongest postcondition* in the Hoare-logic sense for loop-free segments.

Definition 1. *(Strongest Postcondition) For a segment $\mathcal{F}$ and a precondition P, we consider postconditions whose truth, on each feasible path, depends only on the function-entry symbolic values and the endpoint values of locations tracked on that path. Within this scope, a formula Φ is the strongest postcondition of $\mathcal{F}$ with respect to P, if*

- *$\{P\}\,\mathcal{F}\,\{\Phi\}$ holds: every execution under P satisfies Φ at the segment endpoint;*
- *for any postcondition Ψ in this scope such that $\{P\}\,\mathcal{F}\,\{\Psi\}$ holds, we have $\Phi \Rightarrow \Psi$.*

Let $\mathcal{F}$ be a segment from p_{start} to p_{end} with precondition P. Symbolic execution resumes from the incoming symbolic state under P at p_{start}. This state relates the tracked values at the segment start to the function-entry symbolic values, which remain the basis of subsequent expressions. Let $\Pi = \mathit{Path}(p_{start}, p_{end})$ be the feasible paths explored under P, and $\Phi_{SP} = \mathit{SP}(\mathcal{F}, P, p_{start}, p_{end})$. We define

$$\Phi_{SP} = \bigvee_{\pi \in \Pi} \big(PC_\pi \wedge \rho(\sigma_{p_{end},\pi})\big).$$

Each disjunct uses the full symbolic store for its path; the tracked location sets may differ across paths.

Proposition 1. *$\mathit{SP}(\mathcal{F}, P, p_{start}, p_{end})$ is the strongest postcondition of $\mathcal{F}$ with respect to P, if the following conditions hold:*

(A1) Scope conditions: $\mathcal{F}$ contains no external calls (i.e., every called function has a definition), no loops, no recursive datatypes, and no recursive function calls. Every memory access is representable in the flattened model. For every input satisfying P, execution of the interval is well-defined under the modeled C semantics: memory accesses, aliasing, and arithmetic operations are represented exactly, and no undefined behavior occurs.

(A2) Symbolic execution conditions: The incoming symbolic state exactly characterizes the segment-start states under P and their relation to the function-entry values. Symbolic execution is path-complete: it enumerates all feasible paths from p_{start} to p_{end} under P. For each path π, PC_π exactly characterizes the inputs that cause execution to follow π, and $\sigma_{p_{end},\pi}$ exactly expresses the resulting values of all locations tracked on that path in terms of the function-entry symbolic values.

The proof is given in Appendix A.

Unsupported features cause *SP* to return *SymExecFailed* and invoke the LLM fallback; undefined behavior instead produces an error. Fallback specifications may be verifier-checked but do not inherit the strongest-postcondition guarantee of Proposition 1.

B. Default Precondition Generation

$\mathrm{DefaultPre}(\mathcal{F})$ generates sufficient preconditions for the supported symbolic-execution model. It emits three kinds of facts: validity (each pointer points to allocated memory), separation (distinct collected pointers point to initially disjoint memory regions), and bounds (restricting array lengths and integer values to the configured analysis ranges, e.g. $0 < \mathit{arr_len} < \mathit{MAX_LEN}$ and $0 \le v \le \mathit{MAX_VAL}$ for integer value v). These restrictions define the input domain of the generated contract.

The generator processes the function parameters: for each pointer (including those inside structs and arrays), it emits a validity fact; after all pointers are collected, it appends a single separation fact asserting that their initial memory regions are pairwise disjoint. Validity and bounds help exclude common null-pointer and out-of-bounds runtime errors. QCP requires explicit initial memory predicates to justify dereferences, so the default separation fixes one layout rather than enumerating alias cases (Appendix B). Supported aliasing configurations can be specified by replacing the relevant default separation clauses with appropriate aliasing predicates. A function's actual specification therefore depends on its calling context and actual arguments. The default generator does not emit additional functional assumptions such as sortedness.

After any such replacement, additional designer-supplied precondition clauses are conjoined with the resulting precondition to form PC_0 at p_0 before symbolic execution begins. Fig. 1 illustrates both modes: the validity and separation facts on `CheckCalFun` are auto-generated, while the range constraints on its array values and length field are designer supplied.

In contrast, for recursive pointer-linked data structures (linked lists, trees, recursive structs), the number of reachable memory cells is determined only at runtime (by the list length or tree depth). Therefore, the static generator cannot emit a finite set of validity/separation facts for the entire shape; instead, the shape must be captured by an inductive predicate. In this case $\mathrm{DefaultPre}$ delegates the generation of the inductive shape predicate to the LLM fallback layer (Section III-D), which proposes the predicate and the corresponding precondition. The non-recursive part (validity of the head pointer and of primitive fields) is still emitted by the static generator. The LLM-generated predicate and precondition clauses are kept only if Frama-C/WP accepts them.

C. Function Specification Generation Algorithm

Given a function $\mathcal{F}$, a call stack *cS*, and a collection of annotated functions *aF*, $\mathrm{FuncSpec}$ returns $\mathcal{F}$ annotated with preconditions, postconditions, and loop invariants. Its purpose is to derive a function-level behavioral specification by composing symbolic summaries of loop-free fragments with callee specifications and loop invariants.

Algorithm 1: Function Specification Generation $\mathrm{FuncSpec}$

Input: function $\mathcal{F}$, call stack *cS*, annotated functions *aF*
Output: function $\mathcal{F}$ with specification $\mathcal{S}$

if $\mathcal{F} \in aF$ **then**
  return
cS.push($\mathcal{F}$);
$lS \leftarrow \mathrm{InitLoopStack}()$;
$P \leftarrow \mathrm{DefaultPre}(\mathcal{F})$;
$p_{start} \leftarrow p_0$;
$p \leftarrow \mathrm{SEStart}(\mathcal{F}, P, p_{start})$;
while $p \neq p_{-1}$ **do**
  $P \leftarrow \mathit{SP}(\mathcal{F}, P, p_{start}, p)$;
  $p_{start} \leftarrow p$;
  if $p = p_{loop}$ **then**
    $\mathcal{F} \leftarrow \mathrm{LoopInvGen}(\mathcal{F}, p, lS, cS, aF, P, \mathit{LLM})$;
  if $p = p_{callee}$ **then**
    $\mathcal{F}_{callee} \leftarrow \mathrm{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$;
  $p \leftarrow \mathrm{SEStart}(\mathcal{F}, P, p_{start})$;
$\mathcal{S} \leftarrow \mathit{SP}(\mathcal{F}, P, p_{start}, p_{-1})$;
$\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{S})$;
aF.append(cS.pop());
return $\mathcal{F}$;

The process begins by constructing a default precondition (Section III-B) and initializing the symbolic state at the function

entry. Symbolic execution then proceeds through loop-free regions, summarizing their path conditions and memory updates at the next loop entry, function call, or function exit. At a loop entry, the accumulated symbolic facts guide invariant generation; at an ordinary function call, the callee specification is reused if available or generated before execution continues. This bottom-up organization allows caller analysis to use callee specifications and enclosing-loop analysis to use annotations for nested loops. At the function exit, the final symbolic facts are aggregated into a postcondition, and the annotated function is recorded for reuse.

Two design choices support this composition. First, loop handling remains grounded in symbolic analysis: loop-entry facts and information about variable updates constrain the invariant templates, while the LLM supplies the missing predicates and verifier feedback guides refinement (Section IV-B). Second, the LLM fallback layer (Section III-D) surrounds this core procedure and handles cases where a usable symbolic summary cannot be obtained, including self-recursive functions, recursive data structures, and external calls. The symbolic-execution-driven core is shown in Algorithm 1.

D. LLM Fallback

The LLM fallback wraps Algorithm 1 and has two triggers: (i) self-recursion or recursive data structures, routed at function entry because the executor neither computes recursive fixpoints nor enumerates unbounded shape cells; and (ii) *SP* returning *SymExecFailed* at function exit, covering external calls, path explosion, runtime errors, and other unsupported program features. Loops and ordinary callees instead use $\mathrm{LoopInvGen}$ and recursive $\mathrm{FuncSpec}$; their verifier failures go to the separate refinement layer.

To replace a missing symbolic summary, the fallback LLM uses the code, available symbolic states and path conditions, prior callee and loop summaries, and an ACSL skeleton to propose preconditions, assigns clauses, postconditions, and auxiliary inductive predicates. Prompt and task details appear in Appendix C.

Fallback candidates are retained only if Frama-C/WP proves them under the configured memory model and solvers; failures enter refinement (Section IV-C). Accepted candidates can cover cases outside the exact symbolic-execution fragment as sound over-approximations, but lack the strongest-postcondition guarantee of Proposition 1.

IV. Loop Invariant Generation

The general loop invariant generation problem can be defined as follows:

Definition 2. *Given a loop l : `while B do C` of program $\mathcal{P}$, a precondition P that holds at entry l, and an (optional) postcondition Q, the loop invariant generation problem is to find a logical formula Inv that satisfies the following conditions:*

- ***Base:*** *$P \Rightarrow Inv$, i.e., the invariant Inv holds initially;*
- ***Preservation:*** *$\{Inv \wedge B\}\ C\ \{Inv\}$, i.e., if Inv holds at the start of a loop iteration and loop condition B is true, executing loop body C preserves Inv;*
- ***Termination:*** *$Inv \wedge \neg B \Rightarrow Q$, i.e., upon loop termination, B turns false, Inv remains true and implies postcondition Q.*

When Q is provided, the invariant must also prove that target; without Q, SESPEC still seeks informative invariants. Symbolic states supply template guards, variable classifications, frame facts, and relation placeholders for LLM completion. Section V-E evaluates the resulting invariants.

A. Invariant Generation for Loops

Given a C function, a target loop, its symbolic entry precondition, and an LLM, $\mathrm{LoopInvGen}$ returns the function annotated with a candidate invariant. Its aim is to find a property that holds at loop entry and is preserved by every iteration, thereby supporting verification of the enclosing function.

Generation has three main stages: symbolic analysis, template-based synthesis, and verification-guided refinement. After the LLM examines the loop's behavior, symbolic execution analyzes a single-iteration unfolding to identify unchanged variables and update patterns. The resulting classifications and loop-entry facts constrain invariant templates, while the LLM supplies the predicates needed to summarize repeated iterations. The candidates are inserted into the function and checked by the verifier; failures trigger refinement using both verifier feedback and symbolic context.

A key design choice is to process nested loops from the innermost outward. At the outer-loop boundary, a suitable symbolic entry precondition for an inner loop is not yet available, so its candidate invariant is generated directly from code context without a symbolic precondition or template. The outer loop can then use the annotated body and symbolic information to construct its templates. The complete algorithms are given in Appendix D.

B. Template Generation

$\mathrm{LoopInvTemGen}$ constructs loop-invariant templates from the loop body, precondition, and sets of unchanged and non-inductive variables. It instantiates the templates with path conditions and symbolic entry values obtained from symbolic execution, leaving placeholders for predicates to be synthesized, such as $\Phi(v)$. $\mathrm{OuterLoopLLMCall}$ then passes these templates, together with the program context and loop precondition, to the LLM, which fills the predicate placeholders to produce candidate loop invariants (Algorithm 3 in Appendix D). Prompt examples and ACSL guidance are described in Appendix E-B. The six templates below cover four *SP*-derived variable forms, one form enabled by a supplied verification goal, and a free-form *sink*. The sink accommodates quantified array relations and inductive predicates that symbolic execution cannot anchor.

- Unchanged variables: For any variable $v \in uV$ and any execution path π_j to the loop entry p, v retains its symbolic value $\sigma_{p,\pi_j}(v)$ at the loop entry:

$$PC_{\pi_j} \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- Loop skip conditional property: For a path π_j to p and a loop condition B, if B for path π_j is false, the loop is skipped without altering variables:

$$\neg(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- Non-inductive variables: For a non-inductive variable v (whose value is updated independently of its value from the previous iteration) and any path π_j to p:

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow (v = \sigma_{p,\pi_j}(v)) \vee \Phi(v)$$

  where $\Phi(v)$ is the inductive invariant to be synthesized.
- Inductive variables: For an inductive variable v (which is neither unchanged nor non-inductive) and any path π_j to p:

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(v)$$

- Verification goals: In addition to variable-specific templates, we also introduce templates corresponding to each verification goal. For a verification goal g and any path π_j to p:

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(g)$$

- Free-form sink: an unanchored slot Φ that lets the LLM propose invariants involving quantifiers and inductive predicates that the deterministic forms above cannot anchor.

Symbolic templates turn invariant synthesis into constrained completion: they provide ACSL structure and path-sensitive facts while leaving relations for the LLM. Verifier-guided refinement then retains clauses accepted by Frama-C/WP.

C. Verifier-Guided Specification Refinement

Candidates may fail to parse, fail Frama-C/WP proof, or prove too weak to discharge a supplied assertion. Refinement takes the annotated program $\mathcal{F}$, editable annotation region $\mathcal{A}_0$, saved symbolic context Γ, target kind $k \in \{loop, spec\}$, and LLM; the rest of the program $\mathcal{F}^-$ stays fixed. It returns a verifier-checked $\mathcal{A}^*$ in $\mathcal{F}^*$, or the strongest verified subset found within the iteration budget.

Algorithm 4 (Appendix F) verifies each candidate while editing only $\mathcal{A}$. Syntax/type errors trigger syntax repair; proof failures are classified by role and k, and only present categories enter the next prompt. Success stops the loop; budget exhaustion triggers conservative Houdini pruning.

Refinement has three layers:

Layer 1: syntax repair. For syntax or type errors, the LLM receives only the annotation region and verifier error; Frama-C/WP rechecks its repair before semantic proof.

Layer 2: strategy-driven semantic refinement or regeneration. For proof failures, SESPEC selects strategies for the observed obligation types (loop, function, callee, frame, or assertion; Table VII, Appendix F). The LLM sees each failure with its matching strategy and revises invalid clauses, strengthens insufficient ones, or regenerates a globally inconsistent region.

Layer 3: Houdini pruning. At budget exhaustion, one Houdini pass removes verifier-identified failing clauses: loop invariants for loop refinement, or target `ensures` clauses and residual invariants for function refinement. It returns the strongest verifier-checked subset found.

The loop and function versions differ only in the error categories selected by $\mathrm{Classify}(Errors, k)$. The implementation also retains the best candidate by the lexicographic score

$$\mathrm{Score}(\mathcal{F}) = \big(\mathbf{1}_{\mathrm{parse}},\ \tfrac{|\mathrm{Valid}(VC)|}{|VC|},\ |C|\big),$$

Here $\mathbf{1}_{\mathrm{parse}}$ is one iff the candidate parses, VC is the candidate's verification conditions, $\mathrm{Valid}(VC)$ the proved subset, and C its retained clauses. The score prefers parseability, then proof ratio, then clause count. After two non-improving rounds (*patience* $= 2$), refinement resumes from the best candidate; Houdini also starts there at budget exhaustion.

V. EXPERIMENTS

This section evaluates SESPEC on benchmark families from prior invariant and specification generation work, which enables direct C/ACSL comparisons under established settings. Section VI complements this with a separate goal-independent study, where assertions are removed and the generator must decide what functional behavior to specify.

A. Experimental Setup

Benchmark: We evaluate our approach on benchmark families commonly used by recent specification and invariant generation tools. These include: **SyGuS** [20], collected by CODE2INV [10]; **OOPSLA** [21]; **NLA**, collected by LIPuS [11]; the datasets from **SV-COMP** and **Frama-C**, collected by AUTOSPEC [13]. In addition, we constructed three new benchmarks to capture more program scenarios: the **pIp** set consists of small C fragments adapted from a navigation component of an industrial control codebase, involving pointer and struct manipulations and branching; these fragments form a small, self-contained industrial sample adapted for Frama-C/QCP; the **list-S** set is derived from the LIG-MM [22] benchmark and includes all singly linked list programs from SLING and SV-COMP; and the **numer-S** set is a curated collection of additional programs, comprising 20 linear and 20 nonlinear loops, primarily drawn from CLAUSE2INV [12] and harder than those in **SyGuS** and **NLA**.

Verifier: All experiments are performed using Frama-C version 27.1 (Cobalt). We employ the WP plugin as the verification backend, which enforces a 10-second timeout per query, uses Z3 as the primary SMT solver, and applies the Typed memory model.

Model: We evaluate four LLMs of increasing capability: **GPT-3.5-turbo**, **GPT-4o-mini**, **GPT-4o**, and **Claude 3.7 Sonnet**. For all experiments, we use a fixed sampling temperature of 0.7 to balance output diversity and stability.

B. RQ1: Which program classes can SESPEC handle?

Table I classifies 395 programs by control flow and data type; blue cells need loop invariants, while red cells also need function contracts. SESPEC verifies 353/395 (89.4%), including pointer/struct and linked-list cases. It reaches about 97% on linear-loop and iteration-free programs but 77% on

TABLE I
DATASET OVERVIEW BY PROGRAM TYPE.

Program Type	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S	#Programs	#Solved
Linear Loop	133	30	12	0	20	7	0	0	202	196
Nonlinear Loops	0	0	0	30	20	3	0	0	53	41
Nested & Multi Loop	0	16	7	0	0	1	0	0	24	15
Memory & Loop	0	0	2	0	0	17	12	24	55	45
Loop-free	0	0	0	0	0	20	38	0	58	56
External & Recursive Call	0	0	0	0	0	3	0	0	3	0
#Programs	133	46	21	30	40	51	50	24	395	
#Solved	132	40	16	25	34	39	44	23		353
Avg. LoC	19	24	22	21	19	18	57	24		

nonlinear loops, 63% on nested/multiple loops, and 0/3 on external/recursive calls. Thus lines of code alone poorly predict difficulty; the required invariant or summary matters more.

The weak classes have different causes. At nested or multiple loops, symbolic execution often lacks precise loop preconditions, leaving the LLM to infer relations across phases or loops. Nonlinear programs can require arithmetic facts, helper functions, or inductive definitions beyond the SMT budget. Recursive structures need inductive predicates beyond the finite memory model, while external and recursive calls lack an exact symbolic summary. The LLM fallback covers some simple recursive cases (Appendix J); broader limits are discussed in Appendix K.

> **Finding (RQ1).** SESPEC produces a *Valid* specification for 89.4% (353/395) of programs spanning linear/non-linear loops, memory, and linked lists; the residual failures concentrate on reasoning-heavy cases involving nested/multi-loop relations, non-linear arithmetic, recursive data structures, or external/recursive calls outside the symbolic-execution fragment.

C. RQ2: What syntactic correctness, validity, and accuracy does SESPEC achieve?

We evaluate correctness through three metrics, each using all evaluated programs in the benchmark as the denominator:

- **Syntax rate (Syn.):** the percentage of programs for which the generated specifications are syntactically accepted by the ACSL/Frama-C front end;
- **Validity rate (Val.):** the percentage of programs for which the generated specifications are syntactically correct and all generated clauses are accepted by Frama-C/WP under the verifier configuration in Section V. Pre-existing assertions or verification targets are excluded from Val.; generated loop invariants are valid only if they satisfy both the *Base* and *Preservation* conditions;
- **Accuracy rate (Acc.):** the percentage of programs for which the generated specifications are valid and strong enough to prove the benchmark verification goal. For pre/postconditions, this means implying the given target assertions; for loop invariants, it further requires the *Termination* condition.

All metrics are assessed under Pass@k: a benchmark succeeds if any of up to k complete pipeline attempts (including repair, verification, and refinement; refinement iterations are not counted as separate attempts) satisfies the metric. The Pass@1 cost measurements for the goal-independent study are reported separately in Section VI-D.

TABLE II
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1 AND PASS@5. THE COMPLETE PASS@1/3/5 TABLE IS GIVEN IN TABLE VIII (APPENDIX G).

Benchmark	Model	Pass@1 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.57	63.04	97.83	93.48	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	64.71	50.98	49.02
	GPT-3.5-turbo	76.47	60.78	52.94	82.35	62.75	54.9
	GPT-4o	82.35	72.55	66.67	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	88.00	66.00	92.00	92.00	76.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83

Table II shows that syntax acceptance generally exceeds validity and accuracy, especially for weaker models on complex data. On list-S at Pass@1, GPT-3.5-turbo falls from 70.83% Syn. to 20.83% Val. and 8.33% Acc.: many linked-list annotations parse but fail verification or do not prove the goal. Claude 3.7 Sonnet reaches 95.83%/83.33%/75.00% on the same set.

Model gaps narrow on some families after multiple attempts. At Pass@5, GPT-4o and Claude reach the same Acc. on SV-COMP (76.19%) and NLA (86.67%); on numer-S they

reach 82.50% and 85.00%. This pattern is consistent with symbolic guidance helping a less capable model recover useful candidates, although the table alone does not isolate that effect. Pass@1-to-Pass@5 gains show that extra attempts recover difficult cases, while Pass@3-to-Pass@5 gains are small on NLA, numer-S, and pIp (Appendix G).

The lower Acc. on OOPSLA and SV-COMP is associated with nested or multiple loops, where symbolic execution may not derive a sufficiently strong loop precondition; OOPSLA also lacks explicit loop conditions in some programs. Frama-C cases additionally involve arrays, external calls, and recursion, which challenge the covered symbolic-execution fragment. These mechanisms explain why syntactically accepted annotations can still fail verification or leave a goal unproved.

Finding (RQ2). Syntactic correctness is generally high, but the gap between validity and accuracy widens with benchmark complexity and narrows with model capability; SESPEC narrows model gaps on several benchmark families and reaches most Pass@k gains by Pass@3, with most residual accuracy loss on nested/multi-loop and memory-heavy families.

D. RQ3: How much do symbolic execution and refinement each contribute?

We conduct an ablation study on 100 randomly sampled programs from Table I: 70 loop-only and 30 requiring full specifications. Configurations are token-matched, with at most three refinement iterations; Section VI separately tests the same components on 400 goal-independent programs without that compute cap. Table III compares the LLM alone (Pass@3), symbolic guidance without refinement (Pass@3), and the full pipeline (Pass@1).

TABLE III
ABLATION STUDY.

Method	Syn. (%)	Val. (%)	Acc. (%)
Loop invariant generation (70 programs)			
Only LLM	97.14	37.14	28.57
LLM + sym. exec.	95.71	62.86	60.00
LLM + sym. exec. + refine	95.71	95.71	87.14
Program specification generation (30 programs)			
Only LLM	66.67	26.67	16.67
LLM + sym. exec.	76.67	50.00	46.67
LLM + sym. exec. + refine	93.33	86.67	86.67

Table III shows high syntax rates but low validity and accuracy for the LLM alone. Symbolic guidance raises validity/accuracy by about 28 percentage points on average; refinement then raises loop-only validity to 95.71% and function-level validity from 50.00% to 86.67%. The function-level validity increase is the largest of these gains. Refinement is also the main cost driver, as shown in Section VI.

Finding (RQ3). Symbolic-execution guidance and verifier-guided refinement contribute distinct, complementary gains: over the LLM alone, symbolic execution adds ∼28 pp of validity/accuracy, and refinement then lifts validity to 95.7% on loop invariants and to 86.7% on full specifications.

E. RQ4: Does SESPEC outperform LLM- and SMT-based baselines?

Table IV compares each method's reported effectiveness under its original evaluation protocol: SESPEC permits five complete attempts, whereas AUTOSPEC repeats runs with internal synthesis/verification and ACInv uses augmentation cycles [13], [14]. The number and organization of synthesis and verification attempts therefore differ across methods.

Against numerical-invariant tools, SESPEC is on par with CLAUSE2INV on SyGuS, exceeds LIPuS but trails CLAUSE2INV by 6.6 pp on NLA, and reaches 87% on nested/multiple-loop OOPSLA. These tools specialize in numerical loops; SPECGEN is excluded from this C/ACSL table because it targets Java/JML.

For the C/ACSL LLM baselines, the matched-model comparisons show where the gains arise. On GPT-4o, SESPEC reaches 99.2% on SyGuS versus 86.5% for AUTOSPEC and 79.9% for ACInv; on OOPSLA it reaches 76.1%, matching ACInv and exceeding AUTOSPEC's 71.7%. SV-COMP remains difficult: SESPEC and ACInv both reach 76.2% on GPT-4o. SESPEC generally gains more from stronger models than AUTOSPEC, whose results vary little or decline on several families.

The largest differences occur on programs with pointers and data structures. On pIp, SESPEC reaches 88.0% with Claude 3.7 Sonnet versus AUTOSPEC's 4.0%; the latter often emits invalid ACSL, while symbolic-state-derived skeletons constrain both syntax and memory relations. On linked-list list-S, SESPEC reaches 95.8% with Claude, whereas AUTOSPEC remains at 0%. Inductive-predicate support also helps express properties beyond the finite symbolic-memory model in the Frama-C and list-S cases.

Finding (RQ4). SESPEC matches tools specialized for numerical invariants and clearly outperforms LLM-based generators (AUTOSPEC, ACInv), with the largest margins on the pointer-, struct-, and linked-list families that prior LLM tools handle poorly; its accuracy also scales with model capability where AUTOSPEC's plateaus.

VI. GOAL-INDEPENDENT EVALUATION

The preceding benchmarks often provide `assert` goals. To test goal-independent generation, we remove all such assertions and compare with AUTOSPEC [13] (C/ACSL) and SPECGEN [15] (Java/JML).

A. Experimental Setup

We construct **SESPEC-400** from **FormalBench** [23] (250 Java programs), **SpecGenBench** (59 Java programs), and native-C **SV-COMP** (91). FormalBench originally contains OpenJML-verified Java programs with sequential, branching, and loop-based structures. We translate the Java programs to C while preserving control flow and data dependencies (Appendix H), remove all `assert` statements, and use the Frama-C/WP protocol of Section V (Z3, Typed memory model).

All LLM methods use `gpt-4o`, `gpt-5-mini`, `gpt-5.4-mini`, and `gpt-5` at temperature 0.7 under Pass@1. Using one model family limits cross-family training

TABLE IV
ACCURACY COMPARISON ACROSS BENCHMARKS AND BASELINES. SESPEC REPORTS PASS@5 (CF. TABLE II); BASELINES USE THEIR BEST REPORTED CONFIGURATION.

Method	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S
CODE2INV	54.9%	—	—	—	—	—	—	—
HOLA	—	**93.5%**	—	—	—	—	—	—
CLN2INV	93.2%	—	—	—	—	—	—	—
LIPuS	93.2%	—	—	83.3%	—	—	—	—
CLAUSE2INV	99.2%	—	—	**93.3%**	—	—	—	—
AUTOSPEC-3.5	85.7%	82.6%	**76.2%**	70.0%	75.0%	60.8%	14.0%	0.0%
AUTOSPEC-4o	86.5%	71.7%	66.7%	70.0%	72.5%	68.6%	22.0%	0.0%
AUTOSPEC-Claude 3.7	88.7%	80.4%	66.7%	60.0%	55.0%	74.5%	4.0%	0.0%
ACInv-3.5	71.4%	45.7%	42.9%	—	—	34.5%	—	—
ACInv-4o	79.9%	76.1%	**76.2%**	—	—	48.3%	—	—
SESPEC-3.5	93.2%	47.8%	66.7%	56.7%	67.5%	54.9%	76.0%	16.7%
SESPEC-4o	99.2%	76.1%	**76.2%**	86.7%	82.5%	72.5%	78.0%	79.2%
SESPEC-Claude 3.7	**99.2%**	87.0%	**76.2%**	86.7%	**85.0%**	**76.5%**	**88.0%**	**95.8%**

and capability differences; the models in Section V were fixed by its baselines.

We compare full SESPEC (up to 10 refinement iterations), its *w/o refine* and *Pure LLM* (also without symbolic execution) ablations, and one-shot AUTOSPEC in C/ACSL. As a cross-ecosystem reference, SPECGEN evaluates all 400 cases in Java/JML: 309 original Java programs and Java translations of 91 native-C programs. The ablations isolate symbolic guidance and refinement; only AUTOSPEC shares our verifier setting. Each run has a 1800-second outer timeout.

On all 400 programs, without the per-program compute cap of Section V, we measure validity, strength, cost, and failures. *Valid* means that the verifier discharges every obligation induced by the generated specification within the configured budget. The questions are:

- **RQ5:** How often does SESPEC produce a *Valid* specification?
- **RQ6:** How strong are SESPEC's specifications versus the baselines?
- **RQ7:** What does SESPEC cost in time, tokens, and API calls?
- **RQ8:** What are SESPEC's dominant failure modes?

TABLE V
NATIVE-VERIFIER VALIDITY ON SESPEC-400.

Method	Model	Valid	Valid (%)
AUTOSPEC (C/ACSL)	gpt-4o	115	28.8
	gpt-5-mini	104	26.0
	gpt-5.4-mini	255	63.8
	gpt-5	288	72.0
SPECGEN (Java/JML)	gpt-4o	144	36.0
	gpt-5-mini	161	40.2
	gpt-5.4-mini	198	49.5
	gpt-5	251	62.7
SESPEC w/o refine w/o symbolic execution (Pure LLM)	gpt-4o	52	13.0
	gpt-5-mini	76	19.0
	gpt-5.4-mini	106	26.5
	gpt-5	55	13.8
SESPEC w/o refine	gpt-4o	126	31.5
	gpt-5-mini	107	26.8
	gpt-5.4-mini	132	33.0
	gpt-5	144	36.0
SESPEC	gpt-4o	272	68.0
	gpt-5-mini	333	83.3
	gpt-5.4-mini	373	93.3
	gpt-5	372	93.0

B. RQ5: How often does SESPEC *produce a* Valid *specification?*

Table V shows that full SESPEC has the highest C/ACSL *Valid* rate on every model: 68.0% on gpt-4o, 83.3% on gpt-5-mini, 93.3% on gpt-5.4-mini, and 93.0% on gpt-5. In that order, its gains are +55.0, +64.3, +66.8, and +79.2 pp over Pure LLM, and +39.2, +57.3, +29.5, and +21.0 pp over AUTOSPEC. Refinement alone adds +36.5, +56.5, +60.3, and +57.0 pp, at the cost of repeated LLM and verifier calls.

Without refinement, symbolic guidance raises *Valid* over Pure LLM on every model: from 13.0% to 31.5% on gpt-4o, 19.0% to 26.8% on gpt-5-mini, 26.5% to 33.0% on gpt-5.4-mini, and 13.8% to 36.0% on gpt-5. The unusually low gpt-5 Pure LLM rate is mainly syntactic: its uses of advanced ACSL behaviors and axiomatics often have malformed syntax, causing whole-file parse failures. Supplying local symbolic context before synthesis helps avoid these failures without repeated LLM sampling. For loop-free programs supported by the flattened memory model, symbolic execution can directly compute path-sensitive postconditions without an LLM call.

On `gpt-5.4-mini`, refinement gains saturate near $k = 5$; Houdini adds +14.1 pp, and differential prompts with rollback add about 23 pp over plain error-plus-code repair (Appendix F, Fig. 7).

> **Finding (RQ5).** On **SESPEC-400** the full pipeline reaches 68–93% *Valid*; refinement is the dominant but most expensive source of gain, whereas symbolic-execution guidance is a cheaper one that needs no repeated LLM sampling.

C. RQ6: How strong are SESPEC*'s specifications versus the baselines?*

Validity certifies that a specification is discharged by its verifier, but it does not rank how informative it is. We first compare accepted input domains (preconditions), guarantees on return (postconditions), and predicates at corresponding loop points (loop invariants) independently. More importantly,

we then compare the joint contract formed by pre- and post-conditions, using the standard refinement order: one contract is stronger if its accepted input domain is no smaller and its guarantees on return are no weaker over the common input domain, with at least one strict advantage.

LLM judge and expert review. Separate judge calls compare preconditions by accepted-domain inclusion, postconditions by bidirectional implication over their common input domain, and invariants at corresponding loop points. Fig. 3 also combines the first two dimensions into a contract comparison. Equality requires equality in both the pre-/post-conditions. Missing clauses mean `true`; unestablished implications count as Incomp. Expert review corrected 32 verdicts. Appendix I reports these corrections, prompts, reviewer details, and an A/B-order audit.

Fig. 3 shows stronger SESPEC postconditions and loop invariants. Whereas AUTOSPEC accumulates sampled, verifier-accepted clauses, including redundant or tautological ones, SESPEC supplies local symbolic states to guide path-sensitive and inductive predicates.

AUTOSPEC accepts broader input domains more often (290 versus 96 pairs), while SESPEC has stronger postconditions (342 versus 18) and loop invariants (120 versus 52) more often. In the combined precondition–postcondition contract comparison, SESPEC leads in 172 pairs versus 64 for AUTOSPEC after expert review; WP finds 168 versus 58.

Precondition permissiveness measures input-domain inclusion, not runtime safety. Omitting necessary pointer-validity, bounds, or arithmetic constraints can admit inputs that trigger runtime errors. SESPEC makes these requirements explicit through symbolic execution, so its more restrictive preconditions can serve a safety purpose.

Against SPECGEN (Java/JML), SESPEC leads on loop invariants (183 versus 9), while SPECGEN leads on postconditions (244 versus 100) and combined contracts (186 versus 92). The swapped-order contract counts are tied (129 versus 129); Appendix I gives the full cross-language comparison.

Frama-C/WP cross-check. For the 698 pairs in which both SESPEC and AUTOSPEC produce valid specifications, we use Frama-C/WP to check implications between their preconditions and postconditions and combine the results to compare the joint contracts. The checks use Frama-C 27.1, Alt-Ergo and Z3, with a five-second timeout per proof obligation. SESPEC has stronger contracts in 168 pairs, AUTOSPEC in 58, and 183 pairs are equal. The remaining 289 pairs fall into Other, including conflicting precondition–postcondition directions or unproved implications.

WP assigns a direction only when it proves every corresponding obligation. An unproved implication enters Incomp whether it is false or true but beyond the prover; unsupported harnesses and unresolved loop correspondence also enter Incomp. Timeouts and automation limits therefore leave many invariants unordered. The judge followed by expert review can inspect semantic relations in these unresolved pairs, while WP provides an independent formal check. Fig. 3 confirms the precondition and postcondition trends, but WP orders fewer invariants.

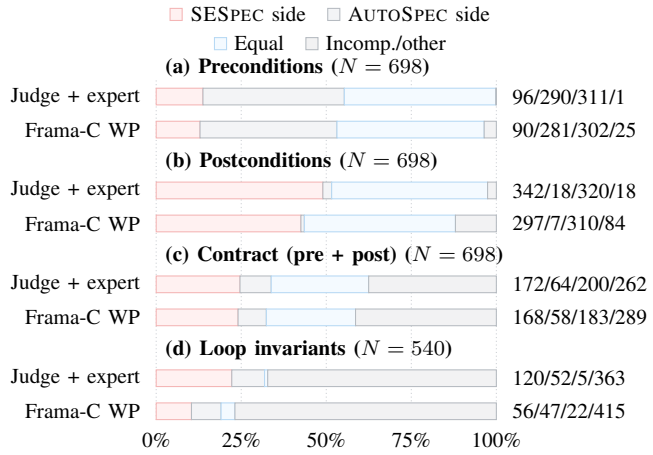


Fig. 3. Dimension-wise relations for AUTOSPEC versus SESPEC over identical data sets. A directional segment means a more permissive accepted domain for preconditions and a logically stronger predicate for postconditions or invariants. For contracts, a side leads when its precondition accepts at least as many inputs and its postcondition is at least as strong, with at least one strict advantage. Equal requires equality in both; conflicting or unresolved combinations enter Other. "Judge + expert" is the corrected judge result; Frama-C WP checks the same implications. Right-hand counts follow the legend order.

> **Finding (RQ6).** AUTOSPEC more often accepts broader input domains, while SESPEC more often leads on guarantees and combined contracts. SESPEC uses symbolic-execution constraints to make validity, separation, and bound requirements explicit in preconditions, while symbolic context guides stronger postconditions and loop invariants.

D. RQ7: What does SESPEC *cost in time, tokens, API calls, and memory?*

Costs are measured in the goal-independent evaluation, with all methods under Pass@1. Fig. 4 compares every attempt, including failures, on `gpt-5.4-mini`; the original AUTOSPEC time, call count, and validity labels are retained. Peak RSS stays below ~ 500 MB.

Each solid (hatched) bar segment sums valid-run (failed-run) costs over all attempts. Failed runs contribute 48.4 of SESPEC's 316.4 seconds per attempt, versus 523.3/933.8 for AUTOSPEC and 235.7/377.2 for SPECGEN. Total time divided by the valid-result count gives approximately 339.3, 1,464.8, and 762.0 seconds per valid specification, respectively. SESPEC's 83.5k tokens per attempt include repeated refinement context without cache discounts.

> **Finding (RQ7).** SESPEC's added framework complexity mainly appears as higher token use; its wall-clock time and API-call count are much closer to the baselines than the raw token totals alone suggest.

E. RQ8: What are SESPEC*'s dominant failure modes?*

Fig. 5 analyzes an independent `gpt-5.4-mini` rerun with per-case logs. Capping refinement at three iterations instead of ten exposes more failures: 321 valid cases versus 373 in the main run. The panels show terminal outcomes and the dominant program feature in each of the 79 non-valid cases (42 *No valid*, 37 *No syntax*); for overlapping features, we choose the first explaining the blocking obligation. Table XIII

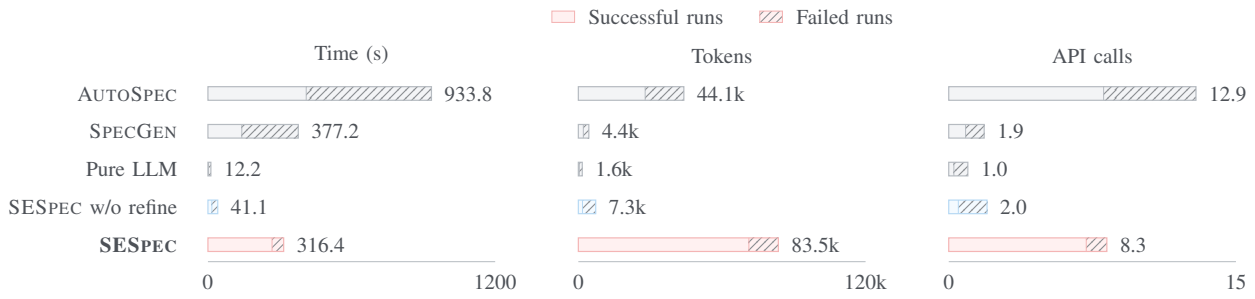
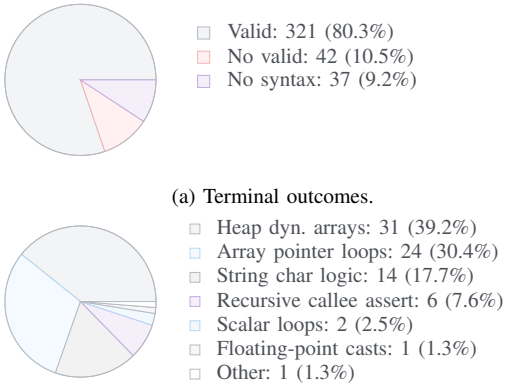


Fig. 4. Mean cost per attempted program with `gpt-5.4-mini`. Each bar stacks the valid-run (solid) and failed-run (hatched) contributions.



(a) Terminal outcomes.

(b) Root causes for non-valid cases.

Fig. 5. Failure outcome and root-cause summary for the `gpt-5.4-mini` SESPEC-400 rerun.

(Appendix K) separately classifies logged syntax and prover events by outcome and obligation type.

Loop invariant obligations account for $2{,}590/4{,}370 = 59.3\%$ of timeouts, especially establishment and preservation over arrays and heap arrays. Postcondition and `assigns` timeouts also occur.

Log outcomes *Timeout*, *Unknown*, and *Failed* denote timeouts, prover uncertainty, and backend or resource failures, not semantic counterexamples. They cannot separate incorrect specifications from correct but unproved obligations. Manual review more often found plausible properties limited by complex loop, array, or heap proofs, without ruling out synthesis errors. See Appendix K for method and verifier limits.

Finding (RQ8). Residual failures frequently involve SMT timeouts on complex proof obligations, particularly those involving loop invariants, arrays, and heap-backed data. Manual inspection further identifies limitations in verifier automation as a major source of the remaining failures.

The ten cases in Appendix J examine phase-split loop invariants, path-complete postconditions under C aliasing, linked-structure shape predicates, RTE safety versus functional specification, inductive helpers, signed arithmetic, recursion, SMT timeouts, and a 225-line industrial control pipeline (13/14 functions specified; 601/625 verification conditions discharged).

VII. Related Work

Traditional approach. Dynamic methods infer candidate invariants from traces: Daikon [9] filters templates against observed tests, Sharma et al. [24] add SMT validation, and DIG [25] infers equations and polyhedra directly. Their results depend on test coverage. Static methods avoid this dependence through constraint solving [2], abstract interpretation [26], [27], interpolation [4], recurrence analysis [28], and abductive or shape reasoning [21], [29]. Their applicability often depends on the target property class or program form; SESPEC uses LLMs to cover the evaluated C/ACSL patterns. These static lines make different trade-offs. Constraint methods can use Farkas' lemma and quantifier elimination to solve for invariant coefficients; polyhedral abstract interpretation uses widening to ensure convergence. Interpolation derives properties from reachability queries, while recurrence methods seek closed forms. Such methods provide precise guarantees within their supported forms but may require templates or manual choices that limit broader applicability.

Machine learning and SMT-based approach. Learning-based invariant generators include CODE2INV [10], which combines graph representations with CEGIS [30] and reinforcement learning; LIPuS [11], which couples RL and SMT with a reward for sparse feedback; and CLN2INV [31], which uses continuous logic networks. Other learning approaches include [32]–[34]. CLAUSE2INV [12] combines LLM-proposed clauses with counterexamples; Loopy [35] filters LLM-generated ACSL invariants with Houdini and error-guided repair. These tools primarily infer goal-driven loop invariants, often for numeric programs, rather than goal-independent functional contracts. Table IV compares the available SyGuS/NLA/OOPSLA results; the numeric tools do not report OOPSLA results.

CODE2INV's reinforcement learning faces sparse rewards; LIPuS explicitly separates candidate generation from SMT checking and changes the reward structure to address that problem. CLAUSE2INV instead generates clauses and combines them using counterexamples, and Loopy moves this style of search to ACSL annotations checked by Frama-C. Despite their different search strategies, a supplied verification goal guides the invariant they seek. This matters for our setting, where the tool must choose functional properties to specify even when no assertion is present.

LLM-based approach. Pei *et al.* [36] fine-tune LLMs to predict invariants without formal guarantees. SPECGEN [15] generates Java/JML candidates and repairs failures through

TABLE VI
POSITIONING OF SESPEC AGAINST REPRESENTATIVE SPECIFICATION AND INVARIANT GENERATION TOOLS.

Tool	Target	Goal	Method	Program Support	Array/Struct/Rec.	Validation
Daikon	invariants	test-driven	traces/templates	Java/C/C++ traces	support/no opt.	trace filter
LIPuS	loop inv.	goal-driven	SMT/RL clauses	C loops	unsupported	SMT
CLN2INV	loop inv.	goal-driven	continuous logic nets	C loops	unsupported	SMT
CLAUSE2INV	loop inv.	goal-driven	LLM clauses + CEX	C loops	unsupported	SMT
Loopy	loop inv.	goal-driven	LLM + Houdini repair	C loops	unsupported	Frama-C
AUTOSPEC	ACSL specs	goal-driven	decomposition + few-shot	C function/loops	support/no opt.	Frama-C
ACInv	ACSL specs	goal-driven	static abstractions	C function/loops	optimized	Frama-C
Preguss	ACSL specs	goal-driven	RTE alarms	C function/loops	support/no opt.	Frama-C
SPECGEN	JML specs	goal-independent	LLM + mutation	Java function/loops	support/no opt.	OpenJML
SESPEC	ACSL specs	goal-independent	symbolic states/templates	C function/loops	optimized	Frama-C

mutation. Preguss [16] uses abstract-interpretation RTE alarms to synthesize minimal safety specifications. For C/ACSL, AUTOSPEC [13] decomposes programs and prompts with examples, while ACInv [14] feeds loop and data-structure abstractions to the LLM. Section V compares SESPEC with AUTOSPEC and ACInv.

These differences delimit the scope of the experimental comparisons: AUTOSPEC and ACInv can be compared with SESPEC in the same C/ACSL verification setting, whereas results from SPECGEN provide cross-language context.

Compared with the loop and data-structure abstractions supplied by ACInv, SESPEC's symbolic states additionally preserve path conditions, memory updates, alias distinctions, and field correlations to guide path-sensitive functional specification generation. For code outside the exact symbolic-execution fragment, SESPEC uses LLM fallback to generate candidate specifications and refines them using verifier feedback. Preguss targets runtime-error freedom; SESPEC generates functional summaries, uses RTE reports as warnings or to trigger fallback when execution is blocked, and may strengthen safety preconditions during refinement.

Lean-based specification generation. CLEVER [37] and VERINA [38] generate specifications and proofs for programs written in Lean. Lean supports expressive inductive predicates and proofs, but applying these systems to existing C code requires rewriting the implementation. SESPEC instead attaches ACSL contracts to native C.

Table VI summarizes these differences. SESPEC generates candidate functional specifications for native C without a supplied verification goal and checks their correctness with a formal verifier.

VIII. CONCLUSION

This paper presents SESPEC, a framework for automated C program specification generation that integrates symbolic execution, large language models, and formal verification. It decomposes the generation task at program boundaries, guides candidate specification generation with path-sensitive symbolic states, and refines the candidates using verifier feedback, thereby supporting functional specification generation without a supplied verification goal.

Experiments across benchmarks covering numerical computation, pointers, structs, and linked lists demonstrate SESPEC's applicability to different classes of C programs, with particularly notable gains on programs involving pointers and data structures. The SESPEC-400 evaluation further demonstrates its ability to generate valid specifications without a supplied verification goal. In the comparison with AUTOSPEC on this benchmark, SESPEC more often provides the stronger overall contract among pairs in which both specifications are valid, a trend also supported by the Frama-C/WP cross-check.

The remaining failures concentrate on inductive, recursive, and heap-heavy proof obligations, where the current Frama-C/WP backend encounters difficult automation cases. As analyzed in Appendices K and L, these cases identify richer verification support as an important direction for extending the approach. Future systems could use an LLM agent to schedule symbolic execution, strengthen specifications, and introduce lemmas while continuing to ground each candidate in symbolic states and verifier feedback.

DATA AVAILABILITY

All datasets, implementation code, experimental results, and supplementary materials related to this work are publicly available at https://github.com/taylor-swift-13/SESpec.

REFERENCES

[1] M. Colón, S. Sankaranarayanan, and H. Sipma, "Linear invariant generation using non-linear constraint solving," in *Computer Aided Verification CAV*, ser. Lecture Notes in Computer Science, vol. 2725. Springer, 2003, pp. 420–432.

[2] H. Liu, H. Fu, Z. Yu, J. Song, and G. Li, "Scalable linear invariant generation with farkas' lemma," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 204–232, 2022.

[3] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanán, and H. Hansen, "FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 793–803.

[4] T. Gan, L. Dai, B. Xia, N. Zhan, D. Kapur, and M. Chen, "Interpolant synthesis for quadratic polynomial inequalities and combination with EUF," in *Automated Reasoning - 8th International Joint Conference, IJCAR*, ser. Lecture Notes in Computer Science, vol. 9706. Springer, 2016, pp. 195–212.

[5] T. Gan, B. Xia, B. Xue, N. Zhan, and L. Dai, "Nonlinear Craig Interpolant Generation," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 415–438.

[6] D. Beyer, M. Kettl, and T. Lemberger, "Decomposing software verification using distributed summary synthesis," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1307–1329, 2024.

[7] D. Wonisch and H. Wehrheim, "Predicate analysis with block-abstraction memoization," in *International Conference on Formal Engineering Methods*. Springer, 2012, pp. 332–347.

[8] C. Calcagno, D. Distefano, P. O'Hearn, and H. Yang, "Compositional shape analysis by means of bi-abduction," in *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 2009, pp. 289–300.

[9] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Sci. Comput. Program.*, vol. 69, no. 1, pp. 35–45, 2007, special issue on Experimental Software and Toolkits.

[10] X. Si, A. Naik, H. Dai, M. Naik, and L. Song, "Code2Inv: A Deep Learning Framework for Program Verification," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 151–164.

[11] S. Yu, T. Wang, and J. Wang, "Loop Invariant Inference through SMT Solving Enhanced Reinforcement Learning," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. Association for Computing Machinery, 2023, p. 175–187.

[12] W. Cao, G. Wu, T. Xu, Y. Yao, H. Wei, T. Chen, and X. Ma, "Clause2inv: A generate-combine-check framework for loop invariant inference," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1009–1030, 2025.

[13] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, and C. Tian, "Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification," in *Computer Aided Verification: 36th International Conference, CAV 2024, Montreal*. Springer-Verlag, 2024, p. 302–328.

[14] R. Liu, G. Li, M. Chen, L.-I. Wu, and J. Ke, "Enhancing Automated Loop Invariant Generation for Complex Programs with Large Language Models," *CoRR*, vol. abs/2412.10483, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2412.10483

[15] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "SpecGen: Automated Generation of Formal Program Specifications via Large Language Models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, May 2025, pp. 666–666.

[16] Z. Wang, T. Lin, M. Chen, H. Li, M. Yang, X. Yi, S. Qin, Y. Luo, X. Li, B. Gu, L. Lu, and J. Yin, "A tale of 1001 loc: Potential runtime error-guided specification synthesis for verifying large-scale programs," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1874–1902, Apr. 2026. [Online]. Available: http://dx.doi.org/10.1145/3798268

[17] L. Zhou, J. Qin, Q. Wang, A. W. Appel, and Q. Cao, "VST-A: A foundationally sound annotation verifier," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, 2024.

[18] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, N. Zhan, Z. Hu, and Q. Cao, "Qcp: A practical separation logic-based c program verification tool," 2025. [Online]. Available: https://arxiv.org/abs/2505.12878

[19] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, "Frama-C: A software analysis perspective," in *International conference on software engineering and formal methods*. Springer, 2012, pp. 233–247.

[20] R. Alur, D. Fisman, S. Padhi, R. Singh, and A. Udupa, "Sygus-comp 2018: Results and analysis," *CoRR*, vol. abs/1904.07146, 2019.

[21] I. Dillig, T. Dillig, B. Li, and K. McMillan, "Inductive invariant generation via abductive inference," in *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, ser. OOPSLA '13. Association for Computing Machinery, 2013, p. 443–456.

[22] C. Liu, X. Wu, Y. Feng, Q. Cao, and J. Yan, "Towards general loop invariant generation: a benchmark of programs with memory manipulation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 129 120–129 145, 2024.

[23] T. Le-Cong, B. Le, and T. Murray, "Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 21 991–22 014.

[24] R. Sharma, S. Gupta, B. Hariharan, A. Aiken, P. Liang, and A. V. Nori, "A data driven approach for algebraic loop invariants," in *Programming Languages and Systems*. Springer Berlin Heidelberg, 2013, pp. 574–592.

[25] T. Nguyen, D. Kapur, W. Weimer, and S. Forrest, "DIG: A Dynamic Invariant Generator for Polynomial and Array Invariants," *ACM Trans. Softw. Eng. Methodol.*, vol. 23, no. 4, pp. 1–30, 2014.

[26] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '77. Association for Computing Machinery, 1977, p. 238–252.

[27] P. Cousot and N. Halbwachs, "Automatic discovery of linear restraints among variables of a program," in *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '78. Association for Computing Machinery, 1978, p. 84–96.

[28] J. Cyphert and Z. Kincaid, "Solvable polynomial ideals: The ideal reflection for program analysis," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, pp. 724–752, 2024.

[29] T. Nguyen, M. B. Dwyer, and W. Visser, "SymInfer: inferring program invariants using symbolic states," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '17. IEEE Press, 2017, p. 804–814.

[30] A. Abate, C. David, P. Kesseli, D. Kroening, and E. Polgreen, "Counterexample Guided Inductive Synthesis Modulo Theories," in *Computer Aided Verification*. Springer International Publishing, 2018, pp. 270–288.

[31] G. Ryan, J. Wong, J. Yao, R. Gu, and S. Jana, "CLN2INV: learning loop invariants with continuous logic networks," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[32] S. Chakraborty, S. Lahiri, S. Fakhoury, A. Lal, M. Musuvathi, A. Rastogi, A. Senthilnathan, R. Sharma, and N. Swamy, "Ranking LLM-generated loop invariants for program verification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 9164–9175. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.614/

[33] H. Wu, C. Barrett, and N. Narodytska, "Lemur: Integrating Large Language Models in Automated Program Verification," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[34] M. A. A. Pirzada, G. Reger, A. Bhayat, and L. C. Cordeiro, "LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. Association for Computing Machinery, 2024, p. 1395–1407.

[35] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S. K. Lahiri, A. Lal, A. Rastogi, S. Roy, and R. Sharma, "Finding inductive loop invariants using large language models," 2023. [Online]. Available: https://arxiv.org/abs/2311.07948

[36] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, "Can large language models reason about program invariants?" in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023, pp. 27 496 – 27 520.

[37] A. Thakur, J. Lee, G. Tsoukalas *et al.*, "CLEVER: A curated benchmark for formally verified code generation," in *Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025.

[38] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song, "VERINA: Benchmarking verifiable code generation," in *The Fourteenth International Conference on Learning Representations*, 2026.

[39] C. Cadar, D. Dunbar, and D. R. Engler, "KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs," in *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008, pp. 209–224.

[40] V. Kuznetsov, J. Kinder, S. Bucur, and G. Candea, "Efficient state merging in symbolic execution," in *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 193–204. [Online]. Available: https://doi.org/10.1145/2254064.2254088

[41] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley, "Enhancing symbolic execution with veritesting," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, pp. 1083–1094. [Online]. Available: https://doi.org/10.1145/2568225.2568293

[42] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu, "How powerful are LLMs in generating formal program specifications?" in *Forty-third International Conference on Machine Learning*, 2026.

[43] Z. Li, Z. Yang, D. He, H. Zhao, A. Zhao, S. Tang, K. Yang, A. Gupta, Z. Su, and C. Jin, "Goedel-code-prover: Hierarchical proof search for open state-of-the-art code verification," 2026. [Online]. Available: https://arxiv.org/abs/2603.19329

APPENDIX A
PROOF OF THE STRONGEST-POSTCONDITION GUARANTEE

This appendix proves Proposition 1.

Proof. Denote $SP(\mathcal{F}, P, p_{start}, p_{end})$ by Φ. Let s_{entry} denote the function-entry state and s an endpoint valuation. We first establish that Φ characterizes the reachable values on each path's tracked locations, then prove maximality within the scope of the definition.

(Soundness): Consider a concrete execution represented by the incoming symbolic state under P. By (A1), it is well-defined and reaches p_{end} along a feasible path π. By (A2), symbolic execution enumerates this path, PC_π holds, and the symbolic store gives the concrete endpoint value of every tracked location. Thus, the reached valuation satisfies $\ell^{p_{end}} = \sigma_{p_{end},\pi}(\ell)$ for every $\ell \in \mathrm{dom}(\sigma_{p_{end},\pi})$. It therefore satisfies $PC_\pi \wedge \rho(\sigma_{p_{end},\pi})$ and hence Φ.

(Completeness): Let (s_{entry}, s) satisfy Φ. Some disjunct $PC_\pi \wedge \rho(\sigma_{p_{end},\pi})$ holds under this pair. Since PC_π includes P, the function-entry values satisfy the incoming constraints. Incoming-state exactness in (A2) supplies a corresponding segment-start state. By (A1) and the exact path conditions and symbolic stores in (A2), concrete execution follows π to an endpoint valuation t that agrees with s at every location tracked on that path. Thus all tracked values specified by the disjunct are realized by a concrete execution.

(Maximality): Let Ψ be any postcondition within this scope such that $\{P\}\,\mathcal{F}\,\{\Psi\}$ holds. For any (s_{entry}, s) satisfying Φ, completeness supplies a concrete endpoint valuation t on a feasible path π. The Hoare triple gives $\Psi(s_{\text{entry}}, t)$. On π, the truth of Ψ depends only on the function-entry values and the tracked endpoint values. Since s and t agree on these tracked values and share the same function-entry state, $\Psi(s_{\text{entry}}, s)$ also holds. Therefore $\Phi \Rightarrow \Psi$. Under the exact path conditions, Φ itself depends only on the function-entry values and the corresponding path's tracked endpoint values, so it lies within the same scope. Together with soundness, this establishes the strongest-postcondition claim. □

APPENDIX B
DEFAULT SEPARATION AND POINTER ALIASING

QCP needs initial memory predicates that justify dereferences. The assertion `Require emp` describes an empty heap, so symbolic execution of the swap below stops at the first read of `*px`: it cannot derive the precondition of that memory read. A single specification can cover both pointer layouts by stating them as alternatives:

Swap with aliased or disjoint pointers

```
void swap(int *px, int *py)
/*@ With x y
    Require (px == py && x == *px) ||
            (px != py && x == *px && y == *py)
    Ensure  (px == py && x == *px) ||
            (px != py && y == *px && x == *py)
*/
{
    int t = *px;
    *px = *py;
    *py = t;
}
```

In the alias branch, `px == py` and one points-to predicate describe a shared cell. In the other branch, QCP translates the two dereferences into points-to predicates joined by separating conjunction ($*$), which describes disjoint cells. Symbolic execution produces a return proof obligation for each branch; these obligations still require proof. Because a Frama-C/WP contract without separation leaves aliasing unconstrained, representing its valid overlapping and disjoint inputs in QCP would require explicit heap alternatives, so the default precondition generator selects only the disjoint branch to limit symbolic-execution branching. An aliasing input can be supported by replacing the default separation precondition with an appropriate aliasing predicate, as in the first branch above.

APPENDIX C
LLM FALLBACK DETAILS

In the fallback, the LLM acts as a semantic summarization component for the missing symbolic-execution step. It receives the current code fragment, available symbolic states and path conditions, previously generated callee specifications or loop summaries, and an ACSL skeleton. The LLM is tasked with following the relevant paths, tracking changes to variables and memory locations, and proposing ACSL clauses that summarize the observed behavior. The resulting candidates must still be proved by Frama-C/WP.

We use this fallback for four function-level tasks. *Precondition generation* asks the LLM to infer candidate `requires` clauses describing the validity, separation and bound constraints. *Assigns-clause generation* asks the LLM to infer the function's footprint, i.e., `assigns` clauses describing which visible memory locations may be modified. *Postcondition generation* asks the LLM to summarize the relation between the pre-state and the post-state, filling candidate `ensures` clauses. *Inductive-predicate generation* asks the LLM to decide whether the program requires an inductive predicate and, when needed, to propose one for capturing inductive relations in recursive data structures or recursive functions.

APPENDIX D
LOOP ANALYSIS PROCEDURE

Algorithm 2: Loop Analysis LoopAnalysis

Input: loop body $\mathcal{F}_{loop}$, precondition *Pre*, loop stack *lS*, call stack *cS*, annotated functions *aF*, LLM
Output: annotated $\mathcal{F}_{loop}$, sets of unchanged variables *uV* and non-inductive variables *nV*

1 $\mathcal{F}_{unfolded} \leftarrow \text{UnfoldLoop}(\mathcal{F}_{loop})$;
2 $P \leftarrow Pre$;
3 $p_{start} \leftarrow p_{unfolded_0}$;
4 $p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
5 **while** $p \neq p_{unfolded_-1}$ **do**
6 $\quad P \leftarrow SP(\mathcal{F}_{unfolded}, P, p_{start}, p)$; `// Same as FuncSpec`
7 $\quad p_{start} \leftarrow p$;
8 $\quad$ **if** $p = p_{nestedLoopEntry}$ **then**
9 $\quad\quad lS.\text{push}(\mathcal{F}_{nested})$;
10 $\quad$ **if** $p = p_{nestedLoopExit}$ **then**
11 $\quad\quad \mathcal{F}_{nested} \leftarrow lS.\text{pop}()$;
12 $\quad\quad \mathcal{F}_{loop} \leftarrow \text{NestedLoopLLMCall}(\mathcal{F}_{loop}, \mathcal{F}_{nested}, LLM)$;
13 $\quad$ **if** $p = p_{callee}$ **then**
14 $\quad\quad \mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$;
15 $\quad p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
16 ...; `// Same as FuncSpec, construct the final postcondition` $\mathcal{S}$
17 $uV \leftarrow \text{FetchUnchangedVars}(\mathcal{S})$;
18 $nV \leftarrow \text{FetchNonInductiveVars}(\mathcal{S})$;
19 **return** $\mathcal{F}_{loop}, uV, nV$;

LoopAnalysis (Algorithm 2) symbolically executes a single-iteration unfolding $\mathcal{F}_{unfolded}$ of the loop body (Lines 1–4) and then advances through it one program point at a time (Lines 5–15), exposing the update and frame facts of one iteration. The key difference from FuncSpec lies in the handling of nested loops: whereas the outer loop uses symbolic-execution-derived preconditions and templates to guide LLM synthesis (OuterLoopLLMCall), an inner loop is processed purely through NestedLoopLLMCall (Lines 10–12), which infers its invariant directly from the code context without preconditions or templates, because a well-defined precondition is not available at the enclosing loop boundary. Nested loops are thus discharged bottom-up using the loop stack *lS*, while function calls are handled consistently with FuncSpec through the call stack *cS* (Lines 13–14): the callee is summarized before symbolic execution of the unfolded body resumes. Once the unfolded body has been fully executed, its final postcondition is constructed and all nested loops are annotated with the invariants generated for them; this postcondition is then used to classify the unchanged variables *uV* and non-inductive variables *nV* of the outer loop (Lines 16–18).

The driving LoopInvGen procedure (Algorithm 3) wraps LoopAnalysis with template generation (LoopInvTemGen), a single OuterLoopLLMCall, and a verification-and-refine pass.

Algorithm 3: Loop Invariant Generation LoopInvGen

Input: function $\mathcal{F}$, loop entry p_{loop}, loop stack *lS*, call stack *cS*, annotated functions *aF*, loop precondition P, LLM
Output: function $\mathcal{F}$ annotated with loop invariant $\mathcal{I}$ for the loop $\mathcal{F}_{loop}$ at p_{loop}

`//` $\mathcal{F}_{loop}$ `is the loop body at` p_{loop} `in` $\mathcal{F}$
1 $\text{ThinkInNaturalLanguage}(\mathcal{F}_{loop}, LLM)$;
2 $lS.\text{push}(\mathcal{F}_{loop})$;
3 $\mathcal{F}_{loop}, uV, nV \leftarrow \text{LoopAnalysis}(\mathcal{F}_{loop}, P, lS, cS, aF, LLM)$;
4 $lS.\text{pop}()$; `// lS depth is unchanged across this call`
5 $\mathcal{T} \leftarrow \text{LoopInvTemGen}(\mathcal{F}_{loop}, P, uV, nV)$;
6 $\mathcal{I} \leftarrow \text{OuterLoopLLMCall}(\mathcal{F}, \mathcal{F}_{loop}, P, \mathcal{T}, LLM)$;
7 $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{I})$;
8 **if** $\neg\text{Verify}(\mathcal{F})$ **then**
9 $\quad \mathcal{F} \leftarrow \text{Refine}(\mathcal{F}, \mathcal{I}, P, loop, LLM)$;
10 **return** $\mathcal{F}$;

APPENDIX E
SYSTEM DESIGN AND PROMPT CONSTRUCTION

A. Execution and Verification Backbone

The implementation combines three off-the-shelf components: the QCP symbolic execution engine [17], [18], an LLM, and the Frama-C/WP automated verifier [19], and orchestrates them through a thin translation layer. Symbolic execution drives the pipeline and pauses at each loop and function call, where the LLM proposes the missing annotations from the exposed symbolic state and Frama-C/WP discharges the resulting proof obligations.

The choice of QCP and Frama-C is motivated by their complementary strengths. QCP is used as the symbolic execution engine because it produces Coq-verified postconditions and, more importantly, exposes its symbolic state (path conditions plus symbolic stores) at arbitrary program points in a logical form that is directly reusable for specification synthesis. Crucially, QCP satisfies the symbolic-execution conditions of Proposition 1, thereby providing the foundation for the strongest-postcondition guarantee on loop-free segments. Other widely used alternatives such as Klee [39] are not built on Hoare logic and accordingly neither expose intermediate states at arbitrary locations nor present them in a logic-oriented form suitable for specification synthesis. QCP also has strong support for programs that manipulate complex data structures and ships with a library of verified inductive predicates and lemmas, which matches our goal of producing formally precise specifications. The gap QCP leaves — it requires user-supplied loop invariants and callee pre-/postconditions, and it may emit lemmas that need additional Coq interactive proofs — is exactly the gap filled by the other two components. The LLM proposes the missing invariants and specifications from the symbolic state QCP exposes; Frama-C/WP then discharges the resulting proof obligations automatically through SMT under the LLM-friendly ACSL surface syntax. The three components therefore split the work cleanly: QCP supplies the symbolic semantics, the LLM supplies the specification candidates, and Frama-C/WP closes the verification loop.

Stitching these three components together requires a translation layer, because QCP, the LLM, and Frama-C/WP each speak a different annotation language: QCPL (native to QCP), ACSL (native to Frama-C/WP), and natural-language prompts (for the LLM). The translator maintains a bidirectional bridge between ACSL and QCPL: it translates ACSL clauses into QCPL so that QCP can use them across loop and function boundaries, and it converts QCP's symbolic-state output into

ACSL skeletons (paired with natural-language guidance for the LLM) and ACSL clauses (for Frama-C/WP verification). A deliberate design choice is that the LLM never sees QCPL; it only operates on ACSL plus natural-language guidance derived from the symbolic state. This avoids the common failure mode of confusing two formally similar but semantically distinct languages, and the skeleton-plus-prose discipline ensures structural well-formedness before the LLM fills in the logical content. When verification fails, error information is translated back into natural language, together with its context, and routed into the next LLM call—closing the loop without exposing the verifier's internal representation to the model either.

B. Prompt Construction

Every LLM call in SESPEC follows a fixed template, but the concrete prompt is specialized to the input program. A lightweight static classifier inspects the source code and assigns each program to one of five routes: recursive data structure, recursive program, array, numeric, and plain, where plain is the default route for programs that do not need helper definitions. The selected route determines which worked examples, category-specific ACSL guidance, and route-specific rule fragments are inserted on top of the universal ACSL pitfalls block. Thus, the same template yields substantively different prompts for, say, a numeric recurrence and a linked-list cursor.

This routing drives the five LLM call sites in the pipeline: loop invariant generation, precondition generation, postcondition/assigns generation, syntax repair, and verifier-guided semantic refinement. In addition, the symbolic executor supplies the state-dependent content of each prompt — the variable state and the path constraints — together with an annotation skeleton to be completed by the LLM. At refinement time, the verifier's typed error categories are additionally folded into the prompt. All prompt templates are listed in Section E-C.

C. Full Prompt Templates

This section lists the templates used by the main LLM call sites referenced in Section E-B, abridged to their static skeleton and runtime slots (the verbose in-prompt admonitions are elided). Each template's static body is shown in black; runtime-injected slots appear in gray with colored tags from the shared figure palette: LOOP/SYMBOLIC for loop-aware symbolic context, ROUTE/CALL for route and call-site context, and VERIFIER/SPECIFICATION for verifier errors and specification edits.

1) Loop-invariant generation: The example below is a filled loop-invariant prompt for a small branch-then-loop program, with the long example and guidance blocks elided.

Loop-invariant generation prompt

```
### Role / Task ###
Given a C program with a loop, generate ACSL loop invariants
that let Frama-C verify it.

ROUTE  examples + ACSL helper policy for the matched route, omitted
GUIDANCE  strength / supplementary / assertion guides, omitted

### Rules ###
- Only fill the loop-invariant PLACE_HOLDERs (or add a helper block
  before the function); keep existing annotations unchanged.
- Use the pre-condition, not \at(var, LoopEntry).
- Emit one self-contained ```c``` block.

SYMBOLIC STATE  supplied by the symbolic executor
Pre-condition:
    (\at(a,Pre)==0 && y==0 && x==1 && a==\at(a,Pre))
 || (\at(a,Pre)!=0 && y==0 && x==0 && a==\at(a,Pre))

Loop + skeleton to complete:
/*@
  loop invariant (\at(a,Pre)!=0) ==> PLACE_HOLDER_x;
  loop invariant (\at(a,Pre)!=0) ==> (a==\at(a,Pre));
  ... (symmetric a==0 branch omitted) ...
*/
while (y < 100000) { x = x + y; y = y + 1; }
```

2) Precondition fallback:

Precondition fallback prompt

```
### Role: ###
Generate ONLY function-level preconditions plus supporting
predicate/logic declarations needed by them.

### Goal: ###
Rule out Frama-C runtime errors:
- pointer dereferences must be covered by \valid or
  \valid_read;
- array ranges must cover all indexed accesses;
- inductive data structures use structural predicates from
  examples.

ROUTE EXAMPLES
{examples}

### Output format: ###
Output exactly one fenced ```acsl``` block:
/*@ predicate <name>(...) = ...; */
/*@ logic <type> <name>(...) = ...; */
/*@
  requires <clause>;
*/

### Hard rules: ###
- Output only ACSL annotations.
- Produce requires only: no ensures, assigns, or loop invariant.
- Each predicate or logic declaration lives in its own /*@ ... */
↪ block.

Function under specification:
{code}
```

3) Postcondition and assignment generation:

Postcondition / assignment generation prompt

```
### Task: ###
Transform the following C code into an ACSL-annotated version.

### Rules: ###
- Only fill ensures PLACE_HOLDER and assigns PLACE_HOLDER.
- The input is a complete C file; callee specifications are
  read-only context.
- Output only the modified target function block. The
  harness splices it back into the original file.
- Helper predicate/logic definitions are allowed and required
  if the target specification references a name not already defined.
- Replace the ensures PLACE_HOLDER with as many strong,
↪ non-redundant
  clauses as the behavior admits (typically 3-8): the closed-form
  result, per-branch facts, and frame consequences.
- Inline relations directly in ensures; add a predicate/logic helper
  only when the relation is recursive or higher-order.

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

RESOLVED ARRAY SIZES
{array_length_facts}

### C Program Code: ###
{code}
```

4) Syntax repair: A deterministic ACSL fixer (Section IV-C) runs before this prompt and clears the common mechanical faults; the template below is the LLM fallback used only when the fixer cannot resolve the parser error.

Syntax-repair prompt

```
### Role: ###
You are an expert in C and Frama-C.

### Task: ###
Correct syntactically incorrect ACSL annotations using
Frama-C errors.

Syntax error:
VERIFIER ERROR
{error_str}

C code with incorrect ACSL annotations:
{c_code}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Rules: ###
- Do not modify function bodies or signatures.
- Only change ACSL annotations.
- New top-level predicate/logic blocks may be added before
  the function.
- If an unbound logic/predicate name is reported, define it
  or remove every clause that references it.
```

5) Verifier-guided semantic refinement: Fig. 6 gives the refinement prompt used by the verifier-guided refinement loop of Section IV-C.

Verifier-guided refinement prompt

```
### Task: ###
Frama-C reports verification failures grouped by category. Each
↪ category
has a different fix strategy; only relevant strategies are shown.

### Categorized errors ###
ACTIVE ERROR BUCKETS
{error_str}

### Per-category fix strategy ###
ACTIVE STRATEGIES ONLY
{strategy_blocks}

### Current C code with annotations ###
{c_code}

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Output rules ###
- The target function's requires may be strengthened when necessary.
EDIT MODE
{output_format_block}
- Do not modify user assert statements.
- Do not modify correct loop invariants or correct postconditions.
- Wrap the corrected target block, or the complete file in
↪ full-file mode,
  in one ```c``` fenced block.
```

Fig. 6. Verifier-guided semantic refinement prompt.

APPENDIX F
REFINEMENT PROCEDURE AND BUDGET ABLATION

Algorithm 4 gives the verifier-guided refinement procedure of Section IV-C, and Table VII gives the category-to-strategy mapping used by its strategy-driven layer. Fig. 7 then isolates the effect of refinement budget on gpt-5.4-mini. The cumulative *Valid* rate rises from 24.2% at $k{=}0$ to 60.2% at $k{=}3$, 73.0% at $k{=}5$, and 79.2% at $k{=}10$, and then reaches 93.3% after the final Houdini pass. The marginal gain after $k{=}5$ is smaller, while Houdini contributes a final $+14.1$ pp by pruning failing clauses from the best refinement snapshots. In contrast, the *trivial refine* ablation, which uses only a plain error-plus-code prompt, reaches 70.0% under the same 10-iteration budget and final Houdini pass. This shows that differential refinement prompts and best rollback together account for roughly 23 pp of additional validity.

APPENDIX G
FULL PASS@K BENCHMARK RESULTS

Table VIII supplements Table II (Section V) with the complete Pass@1/3/5 results per benchmark and model.

APPENDIX H
TRANSLATION DETAIL

We used GPT-5.4 to translate the 250 FormalBench and 59 SpecGenBench Java programs into C; the other 91 SV-COMP cases are native C. For each Java case, the model was asked to preserve the target method's branches, loops, assignments, and returns while adapting its types and memory operations to C. We removed JML annotations and goal assertions and renamed the target function to `foo` followed by its case ID. Programs that could not be translated into the supported C fragment and compiled were excluded; for example, SpecGenBench's `ConvertToTitle` uses Java `StringBuffer.append`

Algorithm 4: Verifier-Guided Refinement Refine

Input: $\mathcal{F}$ with candidate annotation $\mathcal{A}_0$, symbolic context Γ, target kind $k \in \{loop, spec\}$, *LLM*
Output: $\mathcal{F}^*$ with verifier-checked annotation $\mathcal{A}^*$

$\mathcal{A} \leftarrow \mathcal{A}_0$;
$\mathcal{F}^- \leftarrow \mathcal{F} - \mathcal{A}_0$;
while *Errors* $\leftarrow \neg$Verify$(\mathcal{F})$ **do**
  if *reach iteration limit* **then**
    `// Layer 3: Houdini-style pruning`
    $\mathcal{A} \leftarrow$ Houdini$(\mathcal{F}^-, \mathcal{A}, Errors)$;
    $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$;
    break;
  if *Errors* $=$ *syntax error* **then**
    `// Layer 1: syntax repair`
    $\mathcal{A} \leftarrow$ RepairCallLLM$(\mathcal{A}, \Gamma, Errors)$;
  else
    `// Layer 2: strategy-driven refinement or regeneration`
    guidance $\leftarrow$ BuildGuide(Classify$(Errors, k), \Gamma)$;
    $\mathcal{A} \leftarrow$ RefineCallLLM$(\mathcal{A}, \Gamma,$ *guidance*$)$;
  $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$;
return $\mathcal{F}$;

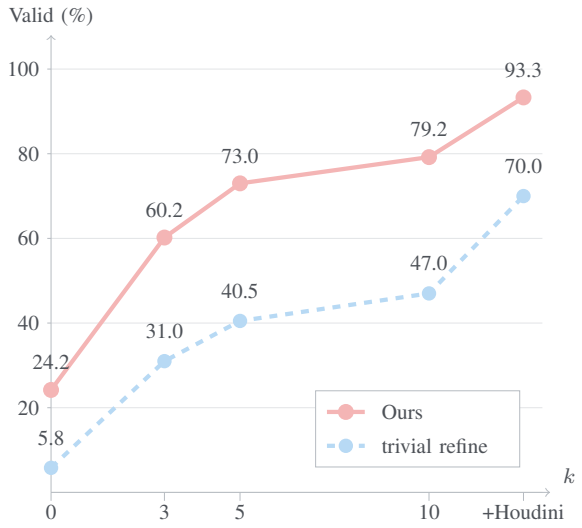


Fig. 7. Cumulative *Valid* rate on gpt-5.4-mini over 400 cases as the refinement budget k increases, for the full SESPEC pipeline (*Ours*) versus a *trivial refine* baseline.

and `reverse` and was not included. All 400 selected C programs pass GCC C11 compilation: 309 translated programs and 91 native-C programs.

For the SPECGEN evaluation, we use the original Java versions of the 309 Java-origin programs and translate the 91 native-C SV-COMP programs into Java. Thus, SPECGEN is evaluated on all 400 corresponding programs, and its validity rates use the same denominator of 400.

In the Java-to-C translations, Java `int`, `long`, and `boolean` are represented by C `int`, `long long`, and 0/1-valued `int`. An array parameter `int[] a` becomes `int *a, int a_len`; `a.length` becomes `a_len`, and array lengths are passed through calls. A local `new int[n]` becomes a C allocation with a stored length. Constants such as `Integer.MIN_VALUE` and the few string operations are adapted to their C counterparts.

Compilation checks syntax, not full semantic equivalence. Cross-language comparisons are restricted to normal executions with valid memory accesses, successful allocation, and no signed overflow.

We also checked translation behavior by differential execution on the 309 program pairs translated from Java to C. The check covered 4,325 comparable executions using deterministic edge-case and seeded pseudorandom inputs, and compared the return value and any modified integer-array arguments. The Java programs were compiled for Java 8, and the C programs were compiled with GCC C11 and undefined-behavior checks. Of these executions, 4,323 produced matching observations; the other two triggered C undefined behavior in the same program. Thus, 308 of 309 pairs showed no discrepancy on their tested inputs, while one pair exposed a cross-language semantic difference.

In that pair, case 112, input `'0'` makes the expression `1 << (c - 'a')` shift by a negative count in C, whereas Java defines the shift and returns 1. This discrepancy is outside the common defined-behavior domain, but it shows why compilation and finite-input agreement cannot establish unrestricted Java/C equivalence.

APPENDIX I
SPECIFICATION STRENGTH COMPARISON

This appendix collects the complete material for the dimension-wise specification-strength study of Section VI-C: the comparison protocol, the judge configuration and prompts, the full pairwise results for both baselines including the cross-language comparison against SPECGEN (Java/JML), and the A/B-order audit of the LLM judge.

A. Comparison protocol

Validity certifies that a specification is discharged by its verifier, but it does not rank how informative it is. We therefore compare accepted input domains (preconditions), normal-return guarantees (postconditions), and predicates at corresponding loop points (loop invariants) independently. Each dimension uses a separate judge prompt and API call. Preconditions are classified by set inclusion between their accepted input domains; postconditions are compared by bidirectional implication over the common input domain; and invariants are compared at corresponding loop points. Thus, the two directional columns in Table IX mean more permissive for preconditions and logically stronger for postconditions and invariants. Fig. 3 also combines the precondition and postcondition verdicts into a contract comparison. A contract is stronger when its accepted input domain is no smaller and its postcondition is no weaker, with at least one strict advantage. Equality requires equality in both dimensions; conflicting or unresolved verdicts enter Other. Loop invariants remain separate. A missing precondition or postcondition denotes `true`. Expert review corrected 32 judge verdicts, shown as transitions in Table IX; any implication that cannot be established is included in Incomp.

TABLE VII
REFINEMENT STRATEGIES BY VERIFIER-FAILURE CATEGORY.

Target	Verifier feedback	Refinement strategy
Loop invariant	Establishment (base) obligation fails	Drop or guard facts not implied by the pre-loop state.
	Preservation obligation fails	Rebuild the inductive relation from the loop body's one-step effect.
	Invariant holds but the downstream goal or assertion is unproved	Add loop-exit consequences linking the negated guard to the inductive and unchanged facts.
	Several invariants fail or the candidate is globally inconsistent	Discard the candidate and regenerate from the original template.
Function specification	Target postcondition obligation fails	Drop over-strong claims or split them into path-sensitive clauses.
	Callee-instance obligation fails at a call site	Strengthen caller-side facts before the call, or weaken an over-strong callee precondition.
	Postcondition holds but a user assertion still fails	Add the missing bridge facts to the specification or to the loop-exit facts.
	Frame (`assigns`) obligation fails	Add only the missing externally visible locations.

B. Judge configuration

We used GPT-5.4 with a 2,048-token output limit, no input truncation, and the provider-default temperature, top-p, and reasoning effort. Each precondition, postcondition, and loop-invariant comparison used a separate API call and the corresponding prompt in Section I-C. The expert review was conducted by two master's-level researchers with formal-verification backgrounds.

C. Pairwise specification strength judges

Fig. 8 gives the three independent judge prompts used for the pairwise comparisons reported in Section VI-C and Table IX. Each invocation receives only the target function's annotation blocks relevant to that dimension; the complete prompt text and structured responses are included in the artifact.

Precondition judge

```
[System]
Compare only accepted input domains. A missing
precondition means true. Report which predicate is
WEAKER, i.e., which one accepts more inputs.
Do not compare postconditions or invariants.

[User]
Evaluate independently:
  B_to_A: P_B implies P_A
  A_to_B: P_A implies P_B
B_to_A only => A_accepts_more
A_to_B only => B_accepts_more
both => equivalent
neither/unknown => incomparable
Return both directions, the relation, and a concise
implication-based reason as strict JSON.
```

Postcondition judge

```
[System]
Compare only normal-return guarantees on the common
accepted input domain. Missing preconditions or
postconditions mean true. Ignore invariant content.

[User]
Let D = P_A AND P_B. Evaluate independently:
  A_to_B: D AND Q_A implies Q_B
  B_to_A: D AND Q_B implies Q_A
A_to_B only => A_guarantees_more
B_to_A only => B_guarantees_more
both => equivalent
neither/unknown => incomparable
Do not reward a wider input domain or infer an
unstated guarantee from the body. Return both
implication directions, relation, and reason as JSON.
```

Loop-invariant judge

```
[System]
Compare only explicit loop invariants at corresponding
program points. Ignore function contracts.

[User]
At each point, conjoin that side's invariants.
Evaluate independently at every point:
  A_to_B: I_A implies I_B
  B_to_A: I_B implies I_A
A_to_B only => A_invariants_stronger
B_to_A only => B_invariants_stronger
both everywhere => equivalent
neither/unknown => incomparable
Do not merge different loop points or infer reachable
states from bodies. Return both implication directions,
relation, and a concise decisive reason as JSON.
```

Fig. 8. Independent strength-judge prompts used before expert review. A and B denote two anonymized specifications; tool identities are not disclosed to the judge.

D. Full pairwise results

Table IX reports the complete dimension and contract results on the both-valid sets for both baselines. The AUTOSPEC (C/ACSL) rows and the Frama-C/WP cross-check are discussed in Section VI-C; the remainder of this subsection analyzes the cross-language comparison against SPECGEN (Java/JML).

Against SPECGEN, SESPEC leads on loop invariants but SPECGEN leads on postconditions and combined contracts. One contributing factor is the difference between the two verification settings: the Java/JML and OpenJML stack supports high-level functional contracts without the pointer-validity and buffer-related obligations required by ACSL and Frama-C/WP. In the C setting, these additional memory-model obligations can make some postconditions difficult for Frama-C/WP to discharge, so SESPEC's verification-driven pipeline may retain fewer postconditions when the corresponding proof obligations cannot be discharged within the allotted budget. The comparison therefore reflects both specification generation and differences in the underlying language and verification stacks.

We omit a WP cross-check for SPECGEN because translating Java/JML reference and array semantics into ACSL would confound the comparison.

E. Position-bias audit (A/B order)

The primary run placed the baseline in A and SESPEC in B. To test order sensitivity, we repeated every judge decision with SESPEC in A and the baseline in B, then mapped the output

TABLE VIII
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1, PASS@3, AND PASS@5 (COMPLETE TABLE).

Benchmark	Model	Pass@1 (%)			Pass@3 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	97.74	95.49	88.72	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	97.74	97.74	92.48	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	43.48	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	65.22	45.65	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	95.65	76.09	73.91	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.57	63.04	97.83	80.43	76.09	97.83	93.48	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	80.95	80.95	42.86	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	71.43	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	90.00	65.00	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	62.75	50.98	49.02	64.71	50.98	49.02
	GPT-3.5-turbo	76.47	60.78	52.94	82.35	62.75	54.9	82.35	62.75	54.9
	GPT-4o	82.35	72.55	66.67	86.27	76.47	70.59	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	82.35	76.47	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	88.00	66.00	92.00	92.00	72.00	92.00	92.00	76.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	72.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	76.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	90.00	88.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	75.00	45.83	41.67	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	87.50	83.33	75.00	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83	100.00	100.00	95.83

TABLE IX
PAIRWISE SPECIFICATION RELATIONS ON BOTH-VALID SETS, REPORTED AS LLM JUDGE → JUDGE + EXPERT REVIEW. INVARIANT COMPARISONS INCLUDE ONLY PAIRS FOR WHICH BOTH TOOLS PRODUCED LOOP INVARIANTS.

Baseline	Dimension	Evaluated	SESPEC side	Baseline side	Equal	Incomp.
AUTOSPEC (C/ACSL)	Preconditions	698	96→96	290→290	311→311	1→1
	Postconditions	698	342→342	18→18	320→320	18→18
	Contracts	698	172→172	64→64	200→200	262→262
	Loop invariants	540	120→120	50→52	5→5	365→363
SPECGEN (Java/JML)	Preconditions	673	161→164	178→174	221→222	113→113
	Postconditions	673	102→100	246→244	218→220	107→109
	Contracts	673	91→92	186→186	106→105	290→290
	Loop invariants	473	183→183	9→9	2→2	279→279

back to the original tool orientation. Table X compares the raw judge results before expert correction. The contract row combines the precondition and postcondition verdicts under the rule in Section VI-C. Pairs with no substantive predicate on either side are deterministically equal and are included in the reported agreement. The principal AutoSpec trends remain stable under the swapped order. For SpecGen, the postcondition and invariant directions persist, while the precondition direction reverses and the contract direction becomes a tie under the swapped order, indicating greater order sensitivity for this cross-language comparison.

TABLE X
A/B-ORDER AUDIT OF THE LLM JUDGE. OUTCOME TUPLES ARE SESPEC/BASELINE/EQUAL/INCOMPARABLE.

Dimension	Pairs	Agreement	Primary	Swapped
AUTOSPEC (C/ACSL)				
Preconditions	698	695 (99.6%)	96/290/311/1	99/288/310/1
Postconditions	698	672 (96.3%)	342/18/320/18	349/11/325/13
Contracts	698	675 (96.7%)	172/64/200/262	187/64/199/248
Loop invariants	540	435 (80.6%)	120/50/5/365	160/49/5/326
SPECGEN (Java/JML)				
Preconditions	673	546 (81.1%)	161/178/221/113	200/86/232/155
Postconditions	673	518 (77.0%)	102/246/218/107	141/203/217/112
Contracts	673	539 (80.1%)	91/186/106/290	129/129/110/305
Loop invariants	473	382 (80.8%)	183/9/2/279	198/3/1/271

Case 1: symbolic state exposes loop phases

```
int unknown();

/*@ requires n > 0; */
void SyGuS38(int n) {
  int c = 0;

  /*@
    loop invariant ((c == 0)&&(n == \at(n,Pre))) || (1
     <= c <= n) ;
    loop invariant n == \at(n,Pre);
    loop assigns c ;
  */
  while (unknown()) {
    if(c == n) {
      c = 1;
    }
    else {
      c = c + 1;
    }
  }

  /*@ assert (c == n) ==> (c >= 0); */
}
```

Fig. 9. Case 1 (SyGuS-38): a nondeterministic loop whose phase structure is exposed by symbolic execution.

Case 2: path-complete postconditions for pointer-manipulating code

```
typedef struct __TripleAbsMax
{
  int abs[3];
  int tmax;
  int* ret;
} TripleAbsMax;

/*@
  requires \valid(pIp);
  requires \valid(pIp->abs+(0..2)) ;
  requires \valid(pIp->ret);
  requires \separated(pIp, pIp->ret);

  ensures pIp->ret == \old(pIp->ret);
  ensures -pIp->abs[2] <= pIp->abs[1] && pIp->abs[0] <= pIp->abs[1] && pIp->abs
       [2] < 0 && pIp->abs[1] >= 0
  && pIp->abs[0] >= 0 ==> pIp->tmax == pIp->abs[1] && *\old(pIp->ret) == pIp->
       abs[1];
  ... all 32 paths

  assigns pIp->tmax, *(pIp->ret);
*/
void TripleAbsMaxFun(TripleAbsMax *pIp)
{
  int absfx1 = pIp->abs[0];
  int absfy2 = pIp->abs[1];
  int absfz3 = pIp->abs[2];

  if (pIp->abs[0] < 0){  absfx1 = -pIp->abs[0];}

  if (pIp->abs[1] < 0){  absfy2 = -pIp->abs[1];}

  if (pIp->abs[2] < 0){  absfz3 = -pIp->abs[2];}

  if (absfx1 > absfy2){  pIp->tmax = absfx1;}
  else{ pIp->tmax = absfy2; }

  if (absfz3 > pIp->tmax){pIp->tmax = absfz3;}

  *(pIp->ret) = pIp->tmax;
}

/*@
  requires \valid(pIp);
  requires \valid(pIp->abs+(0..2));
  requires \valid(pIp->ret);
  requires \separated(pIp, pIp->ret);
*/
void main1(TripleAbsMax *pIp)
{
  pIp -> abs[0] = 1;
  pIp -> abs[1] = 2;
  pIp -> abs[2] = -3;

  TripleAbsMaxFun(pIp);

  /*@ assert pIp -> tmax == 3; */
}
```

Fig. 10. Case 2 (TripleAbsMax): a loop-free program with 32 feasible paths under the precondition shown.

APPENDIX J
CASE STUDIES

This appendix presents ten case studies that show, on concrete programs, what symbolic execution contributes to generated specifications. Across all cases, SESPEC reuses the same local symbolic context and template-routing signals, so each case emphasizes a distinct role of that context rather than repeating the full mechanism. Table XI summarizes the cases, which cover loop phases, C aliasing, recursive heap shapes, RTE safety versus functional specification, an industrial pipeline, inductive helpers, signed arithmetic, recursion, and matrix multiplication. In the figures below, ACSL annotations enclosed by $/*@\cdots*/$ are generated by SESPEC and then retained after verification or pruning.

TABLE XI
ORGANIZATION OF THE CASE STUDIES.

Case	Main lesson
1. SyGuS loop	Phase-split invariant from weak goals
2. TripleAbsMax	Path-complete C/ACSL postconditions
3. Linked list	Shape-predicate routing
4. Linear search	RTE-safety vs. functional specification
5. Industrial pipeline	Scale and verifier boundary
6. Factorial	Guarded inductive helper
7. GCD	Recursive summary with a simple measure
8. MulLoop	Sign-sensitive loop invariant
9. Ackermann	ACSL/SMT recursion boundary
10. Matrix multiply	Nested loops; verifier/SMT-timeout boundary

A. Case 1: Phase-Split Invariant from a Weak Goal

Fig. 9 isolates the benefit of symbolic execution when the assertion is too weak to guide invariant synthesis. The goal $(c = n) \Rightarrow (c \geq 0)$ can be proved by many uninformative invariants; what matters here is the loop's phase structure.

At the loop boundary, symbolic execution separates the unentered state (`c == 0`) from the inductive region (`1 <= c <= n`) and records that `n` is stable. The disjunctive template instantiates exactly that split, yielding an invariant that describes the loop behavior rather than merely satisfying the final assertion.

B. Case 2: Path-Complete Postconditions for Pointer-Manipulating Code

Fig. 10, from our **pIp** benchmark, is the main C-specific case. The function computes the maximum absolute value of three struct-stored integers and writes it both to `pIp->tmax` and through `pIp->ret`.

Starting from the default precondition, including `\separated(pIp, pIp->ret)`, symbolic execution derives the guarded postconditions for the feasible paths.

C. Case 3: Shape-Predicate Routing for Linked Structures

Fig. 11 illustrates template routing for a linked-list traversal. The loop walks a singly linked list and returns the original head pointer, so ACSL needs an explicit heap-shape abstraction instead of only arithmetic facts.

Symbolic execution identifies `p` as the advancing list pointer and records the local update facts, while the static route recognizes the recursive type and field-access pattern. That route selects the recursive-shape template, which supplies `lseg`, `listrep`, and `list_length`; the loop template then connects the original head to the current suffix through `lseg(l, p)`. The program-structure route supplies shape

Case 3: route-selected shape predicates

```
struct SLL {
    struct SLL *tail;
    int head;
};

/*@
  predicate lseg{L}(struct SLL* x, struct SLL* y) =
    x == y || (x != y && \valid(x) && \separated(x, y)
     && lseg{L}(x->tail, y));
*/

/*@ predicate listrep(struct SLL* head) = lseg(head, \
     null); */

/*@
  logic integer list_length{L}(struct SLL* x) =
    (x == \null) ? 0 : 1 + list_length{L}(x->tail);
*/

/*@
  requires listrep(l);
  requires \valid_read(&data);
  assigns \nothing;
  ensures listrep(l);
  ensures \result == l;
*/
struct SLL * main1(struct SLL *l, int data)
{
    struct SLL *p;
    p = l;

    /*@
      loop invariant listrep(p);
      loop invariant listrep(l);
      loop invariant lseg(l, p);
      loop invariant data == \at(data,Pre);
      loop invariant l == \at(l,Pre);
      loop invariant list_length(l) >= list_length(p);
      loop assigns p;
    */
    while (p) {
      if (p->head == data) {
        /*@ assert data == \at(data,Pre); */
        /*@ assert l == \at(l,Pre); */
        return l;
      }
      p = p->tail;
    }

    return l;
}
```

Fig. 11. Case 3 (linked-list traversal): a recursive data-structure program with shape predicates.

predicates, and symbolic cursor facts anchor them to this traversal.

D. Case 4: RTE Safety vs. Functional Specification

We reuse the linear search example from Fig. 3 of Preguss [16] and run SESPEC on the same source. The point is not a head-to-head accuracy comparison: Preguss is driven by RTE-safety obligations, while SESPEC attempts to generate goal-independent functional specifications. This distinction also affects failure handling. On the Frama-C validation side, SESPEC records potential RTE reports as warnings rather than treating RTE elimination as the main target. On the QCP side, however, an RTE can still block symbolic execution; the flattened-memory default precondition covers validity, bounds, and separation facts exposed by the model, but it does not guarantee that every arithmetic or pointer-related RTE is avoided. The refinement stage can ask the LLM to strengthen preconditions to avoid such failures, but this is a supporting mechanism for functional specification generation rather than the objective itself.

Preguss: RTE-guided safety specification

```
/*@ requires \valid_read(arr+(0..len-1));
    assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop assigns i; */
    while (i < len) { ... }
}
```

SESPEC: verifier-checked prefix-search specification

```
/*@ logic integer count_eq{L}(int* a, integer lo,
      integer hi, integer v) =
      lo >= hi ? 0 : count_eq(a, lo, hi - 1, v)
                   + (a[hi - 1] == v ? 1 : 0); */
/*@ requires \valid(arr + (0 .. len - 1));
    requires len >= 0;
    ensures \forall integer k; 0 <= k < len ==>
              arr[k] == \at(arr[k], Pre);
    ensures \result == \null ==>
              count_eq(arr, 0, len, val) == 0;
    ensures \result != \null ==>
              count_eq(arr, 0, \result - arr, val) == 0
              && *\result == val;
    assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop invariant 0 <= i <= len;
        loop invariant \forall integer k;
            0 <= k < len ==> arr[k] == \at(arr[k], Pre);
        loop invariant count_eq(arr, 0, i, val) == 0;
        loop assigns i; loop variant len - i; */
    while (i < len) {
        if (arr[i] == val) return arr + i;
        i = i + 1;
    }
    return (int *)0;
}
```

Fig. 12. Case 4 (linear search). Preguss verifies memory-access safety; SESPEC also captures the prefix-search property and returned-pointer consequence.

Fig. 12 shows the difference in target. Preguss emits the memory-safety precondition needed to discharge `arr[i]` accesses. SESPEC instead routes the loop to a count-occurrences helper and records both return cases: null means `val` is absent, and non-null means the returned pointer dereferences to `val` with no earlier occurrence. This illustrates why RTE-safety synthesis of Preguss [16] and our functional specification synthesis should not be read as identical tasks.

E. Case 5: Industrial Pipeline Stress Test

Program structure. We applied SESPEC to a 225-line C transcription of an industrial single-star-sensor Kalman-update pipeline. The translation unit contains 14 functions, 33 call sites, and 8 loops; the entry delegates to three pipeline stages, which in turn call ten numeric and quaternion helpers (Fig. 13). This case is included to show the scale boundary, not another single function invariant pattern.

What SESPEC produced. Driven by `gpt-5.4-mini` at maximum reasoning effort (`reasoning_effort=high`) in the goal-independent configuration, SESPEC completed specification generation for 13 of the 14 functions. The assembled file contains 198 ACSL clauses, and Frama-C/WP discharges 601 of 625 verification conditions (96.2%) at a 30 s per-goal timeout. Table XII shows that increasing the timeout from 5 s to 30 s recovers only one additional goal, so the residual failures reflect hard proof obligations rather than transient budget noise.

To complement the proof-discharge rate, we also manually audited the functional content of each accepted contract. We count a postcondition as functional only when it gives a non-vacuous relation between the output or return value and the inputs; `ensures \true` receives no such credit.

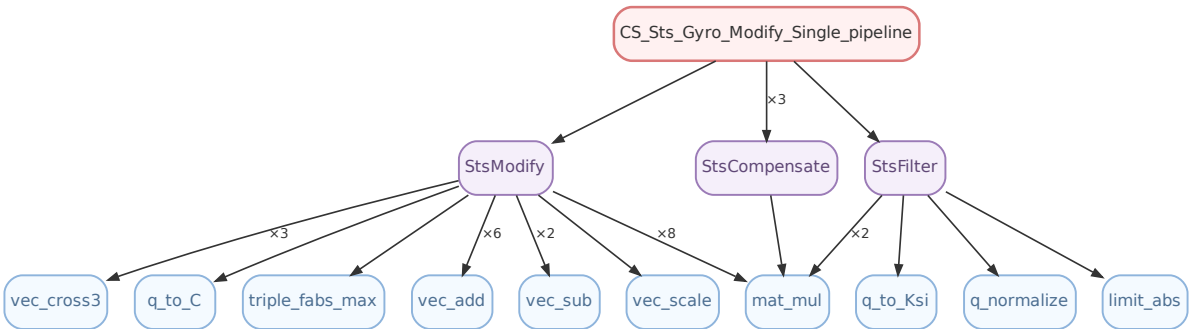


Fig. 13. Call graph of the single-star-sensor filter-update pipeline. Edge labels show call multiplicity; unlabeled edges are ×1.

TABLE XII
FRAMA-C/WP PROOF OBLIGATIONS FOR THE AEROSPACE PIPELINE.

	per-goal timeout		
Disposition	**5 s**	**10 s**	**30 s**
Total	625	625	625
Valid	600	601	601
Unknown	2	2	3
Timeout	23	22	21
Discharge rate	96.0%	96.2%	96.2%

Ten functions have functional postconditions: `limit_abs`, `q_to_C`, `q_to_Ksi`, `triple_fabs_max`, `vec_cross3`, `vec_scale`, `vec_add`, `vec_sub`, `StsCompensate`, and `StsFilter`. The other four lack a complete functional summary: `StsModify` provides only output bounds, `mat_mul` has the vacuous `ensures \true`, and the no-op helper `q_normalize` and the top-level `CS_Sts_Gyro_Modify_Single_pipeline` have no explicit accepted contract.

Boundary exposed. The functions with functional summaries are mostly arithmetic leaves whose semantics admit closed-form ACSL relations, such as `limit_abs`, `vec_cross3`, and `q_to_C`. The hard cases are delegate functions and matrix-style loops: composing multi-stage callee summaries and expressing row-major matrix products require either user-provided logic summaries or proof automation beyond the current SMT-only backend. Together, the 96.2% discharge rate and contract audit characterize both verifier acceptance and functional coverage. Concretely, the undischarged goals concentrate in the matrix-multiply leaf `mat_mul` and its callers, exactly the triple-nested product dissected in Case 10 below, where the recursive `mat_sum` over nonlinear indices makes Z3 time out and the accepted specification degrades to `ensures \true`.

F. Case 6: Guarded Inductive Helper for Factorial

Fig. 14 complements the phase-split loop case (Case 1) above: the loop may be skipped when `n < 1`, and the entered region requires the inductive relation `f == product(1, i - 1)`. Symbolic execution provides the entered/unentered guard, while the static route selects the inductive-helper template that supplies the ACSL `product` definition used by both the invariant and the postcondition.

G. Case 7: Recursive GCD Summary with a Simple Measure

Fig. 15 shows the recursion case where the current workflow succeeds cleanly. When symbolic execution reaches a recursive call, SESPEC switches to the recursion template, emits `gcd_logic` branch-by-branch, and uses `decreases a + b`. This case is useful mainly as a contrast to the Ackermann case below: GCD needs only a single-measure descent and a single-unfold style of reasoning.

Case 6: guarded factorial helper

```
int unknown();

/*@
  logic integer product(integer begin, integer end) =
    (end < begin) ? 1 : product(begin, end - 1) * end;
*/

/*@
  requires \true;
  assigns \nothing;
  ensures (n < 1) ==> \result == 1;
  ensures (n >= 1) ==> \result == product(1, n);
*/
int factorial9(int n) {
  int i = 1;
  int f = 1;

  /*@
    loop invariant (1 <= \at(n,Pre)) ==> (1 <= i <= n + 1);
    loop invariant (1 <= \at(n,Pre)) ==> (f == product(1, i - 1));
    loop invariant (!(1 <= \at(n,Pre))) ==> ((f == 1)&&(i == 1)&&(n == \at(n,
       Pre)));
    loop invariant n == \at(n,Pre);
    loop assigns f, i;
  */
  while (i <= n) {
    f = f * i;
    i = i + 1;
  }

  return f;
}

void goo9() {
  int t = factorial9(5);
  //@ assert t == 120;
}
```

Fig. 14. Case 6 (factorial): a guarded loop invariant with a recursive `product` helper.

H. Case 8: Signed Multiplication Loop

Fig. 16 computes $a \cdot b$ by addition or subtraction depending on the sign of `b`. The negative branch requires the asymmetric invariant `res == -i * a`; reusing the positive-branch form would make the functional postcondition fail. In our run, SESPEC retains `ensures \result == a * b` because the branch condition selects the correct invariant template.

Case 7: recursive GCD summary

```
/*@
  logic integer gcd_logic(integer a, integer b) =
    (a == 0) ? b :
    (b == 0) ? a :
    (a == b) ? a :
    (a > b) ? gcd_logic(a - b, b) :
              gcd_logic(a, b - a);
*/

/*@
requires a >= 0 && b >= 0;
decreases a + b;
assigns \nothing;
ensures \result == gcd_logic(a, b);
*/
int gcd(int a, int b)
{
    if (a == 0)
       return b;

    if (b == 0)
       return a;

    if (a == b)
        return a;

    if (a > b)
        return gcd(a-b, b);
    return gcd(a, b-a);
}

int main2()
{
    int a = 98, b = 56;
    int c = gcd(a, b);
    //@ assert c == 14;
    return 0;
}
```

Fig. 15. Case 7 (GCD): a recursive logic summary with a single arithmetic termination measure.

Case 8: signed multiplication loop

```
int foo57(int a, int b);

/*@
  requires \true;
  assigns \nothing;
  ensures \result == a * b;
*/
int foo57(int a, int b) {
    int res = 0;
    if (b >= 0) {
        /*@
          loop invariant 0 <= i <= b;
          loop invariant res == i * a;
          loop assigns i, res;
          loop variant b - i;
        */
        for (int i = 0; i < b; i++) {
            res = res + a;
        }
    } else {
        /*@
          loop invariant 0 <= i <= -b;
          loop invariant res == -i * a;
          loop assigns i, res;
          loop variant -b - i;
        */
        for (int i = 0; i < -b; i++) {
            res = res - a;
        }
    }
    return res;
}
```

Fig. 16. Case 8 (MulLoop) from **SpecGenBench**.

I. Case 9: Ackermann Recursion Beyond ACSL/SMT Automation

Fig. 17 marks a boundary rather than a prompt failure. The generated helper follows the C branch structure, but Ackermann needs a lexicographic termination argument and nested induction that are not handled well by the current ACSL/SMT stack. A scalar `decreases m * 1000 + n` is not a sound substitute for the true lexicographic descent once `n` grows, and the functional postcondition would require proof support beyond the single-unfold reasoning that succeeds for GCD.

J. Case 10: Matrix Multiplication Beyond Per-Loop Symbolic Context

Fig. 18 is the multi-loop boundary that Reviewer 2's failure-case question targets. SESPEC emits the intended quantified postcondition and the matching outer-loop invariants, while the innermost `s == mat_sum(...,k)` invariant is discharged by one-step induction. The remaining quantified obligations time out in Z3 because they combine recursive logic unfolding, havoc-updated array memory, and non-linear index arithmetic; refinement therefore strips the unproved clauses and the accepted specification collapses to verifier-valid but vacuous `ensures \true`. This case illustrates a proof-automation limit rather than a generation failure.

APPENDIX K
DISCUSSION: LIMITATIONS AND OPEN CHALLENGES

Scope and guarantees. SESPEC targets self-contained programs. Symbolic execution stops at external or system calls, including allocation through `malloc`/`free`, and passes the affected segment to LLM fallback. The strongest-postcondition guarantee in Section III and Proposition 1 applies to covered loop-free segments under path-complete, precise symbolic execution in the flattened memory model. Loops, recursive functions, and recursive data structures receive verifier-checked, best-effort specifications. A discharged specification may still omit functional relations that require unavailable symbolic states, heap abstractions, auxiliary lemmas, or reasoning beyond

Case 9: Ackermann recursion boundary

```
int fool(int m, int n);

/*@
  logic integer ackermann(integer m, integer n) =
    (m == 0) ? n + 1 :
    (n == 0) ? ackermann(m - 1, 1) :
               ackermann(m - 1, ackermann(m, n - 1));
*/

/*@
  requires m >= 0 && n >= 0;
  decreases m * 1000 + n;
  assigns \nothing;
  ensures \result == ackermann(m, n);
*/
int fool(int m, int n) {
    if (m == 0) {
        return n + 1;
    }
    if (n == 0) {
        return fool(m - 1, 1);
    }
    return fool(m - 1, fool(m, n - 1));
}
```

Fig. 17. Case 9 (Ackermann). The candidate specification is syntactically clean but not discharged by Frama-C/WP under the current ACSL/SMT backend.

the configured backend. Allocation modeling and reusable library summaries are future extensions.

Scalability of symbolic execution. Symbolic execution remains difficult to scale to large programs: path explosion and the cost of carrying full symbolic stores both limit it in practice. Many symbolic executors mitigate this: separation logic and shape analysis let them reason about a class of heap-manipulating programs and recursive data structures such as lists and trees [8], while path abstraction and state merging curb path explosion [40], [41]. We deliberately choose a basic symbolic executor instead, with the aim of obtaining precise symbolic states reusable for specification synthesis rather than scaling to larger programs. For what this basic executor cannot cover, we fully leverage the LLM: when the executor reaches a boundary, the framework asks the LLM to continue *in symbolic-execution order*, carrying the latest symbolic state forward and proposing a postcondition or loop invariant consistent with it. This yields stronger specifications than an unguided prompt, but it does not make LLM-generated summaries a sound replacement for symbolic execution beyond the supported fragment.

Verifier-backend ceiling. Two upstream limits cap what can be accepted even when the intended specification is semantically correct. First, some behaviors are cumbersome or unsupported in the ACSL fragment we use, such as lexicographic measures, higher-order quantified properties, and separation-logic-style abstractions over inductive heaps. Even a singly linked-list traversal may require user-defined `lseg`/`listrep` helpers that must be both faithful and SMT-friendly. Second, the ATP backend used by Frama-C/WP cannot reliably automate all quantified, inductive, and memory-heavy obligations generated by these specifications. *Timeout* events observed in our experiments are dominated by loop-invariant obligations, especially preservation and establishment goals over arrays or heap-backed arrays. WP often translates these obligations into quantified formulas over typed-memory stores, address shifts, integer casts, and occasionally recursive logic helpers such as counting functions, so even plausible invariants can exceed the configured per-query solver budget. *Unknown* and *Failed* events observed in our experiments mostly occur in verifier/modeling regions where the backend returns no definitive counterexample: whether the remaining obligations come from specification errors or verifier/modeling limitations remains unresolved. Table XIII counts the logged events underlying this discussion.

Case 10: matrix-multiplication proof boundary

```
/*@
  logic integer mat_sum(int *A, int *B, integer ca_rb, integer cb,
                        integer i, integer j, integer k) =
    k <= 0 ? 0 :
    mat_sum(A, B, ca_rb, cb, i, j, k - 1) +
    A[i * ca_rb + (k - 1)] * B[(k - 1) * cb + j];
*/

/*@
  requires \valid(out + (0 .. ra*cb - 1));
  requires \valid(A   + (0 .. ra*ca_rb - 1));
  requires \valid(B   + (0 .. ca_rb*cb - 1));
  requires \separated(out + (0..ra*cb-1), A + (0..ra*ca_rb-1));
  requires \separated(out + (0..ra*cb-1), B + (0..ca_rb*cb-1));
  // default precondition also bounds every cell to [0,100] and each
  // dimension product to (0,100), keeping the arithmetic overflow-free
  assigns out[0 .. ra*cb - 1];
  ensures \forall integer i, j; 0 <= i < ra && 0 <= j < cb ==>
      out[i*cb + j] == mat_sum(A, B, ca_rb, cb, i, j, ca_rb); // Z3 TIMEOUT (10
       s)
*/
void mat_mul(int *out, int *A, int *B, int ra, int ca_rb, int cb) {
  /*@
    loop invariant 0 <= i <= ra;
    loop invariant \forall integer ii, jj; 0 <= ii < i && 0 <= jj < cb ==>
        out[ii*cb + jj] == mat_sum(A, B, ca_rb, cb, ii, jj, ca_rb); // Z3
     TIMEOUT (10s)
    loop invariant \forall integer p; i*cb <= p < ra*cb ==> out[p] == \at(out[p
      ],Pre); // Z3 TIMEOUT
    loop assigns i, out[0 .. ra*cb - 1];
  */
  for (int i = 0; i < ra; i++) {
    /*@
      loop invariant 0 <= j <= cb;
      loop invariant \forall integer jj; 0 <= jj < j ==>
          out[i*cb + jj] == mat_sum(A, B, ca_rb, cb, i, jj, ca_rb); // Z3
       TIMEOUT (10s)
      loop assigns j, out[i*cb .. i*cb + cb - 1];
    */
    for (int j = 0; j < cb; j++) {
      int s = 0;
      /*@
        loop invariant 0 <= k <= ca_rb;
        loop invariant s == mat_sum(A, B, ca_rb, cb, i, j, k); // DISCHARGED (
       one-step induction)
        loop assigns k, s;
      */
      for (int k = 0; k < ca_rb; k++)
        s = s + A[i*ca_rb + k] * B[k*cb + j];
      out[i*cb + j] = s;
    }
  }
}
```

Fig. 18. Case 10 (matrix multiplication, $ra \times ca_rb$ by $ca_rb \times cb$). SESPEC generates the correct quantified specification, but Frama-C/WP→Z3 times out on the outer invariants and postcondition, so the accepted specification degrades to `ensures \true`.

TABLE XIII
VERIFIER LOG EVENTS FOR THE `GPT-5.4-MINI` SESPEC-400 RERUN.

Prover outcome	Loop Inv.	Post Cond.	Assigns	Other VC	Events
Syntax	—	—	—	—	2,315
Timeout	2,590	697	1,014	69	4,370
Unknown	62	4	111	0	177
Failed	27	22	0	0	49

Together with the program-feature breakdown in Fig. 5, this identifies *the SMT solving capacity of the ATP backend* as the dominant limit on our validation pass rate. Migrating to an interactive theorem prover can make residual obligations dischargeable by allowing explicit proof scripts, and in our manual inspection the remaining obligations become reachable once the LLM writes such scripts under human guidance. This does not eliminate the scalability problem: it shifts the bottleneck from ATP automation to proof construction and proof search. We therefore view ITP integration as a promising future direction for SESPEC, especially as LLMs become better at generating and searching proof scripts, but not as an already solved route to large-scale verification.

APPENDIX L
THREATS TO VALIDITY

We discuss threats to validity along the standard construct, internal, and external dimensions.

Construct validity. Our metrics — syntactic correctness (*Syn.*), validity under Frama-C/WP (*Val.*), and goal accuracy (*Acc.*) — measure proof acceptance under a specific verifier configuration, not semantic adequacy in absolute terms. A verifier-checked specification is not necessarily as informative as a hand-written one, and an unchecked one is not necessarily semantically wrong: it may be unprovable under the ACSL fragment, memory model, or ATP backend discussed in Appendix K. Quality is also hard to compare when two valid specifications differ in strength, readability, framing precision, or coverage. To partially address this, the experiment in Section VI adds an LLM-as-judge pairwise strength comparison on the both-valid intersection. This comparison is itself approximate: the judge is an LLM, may weight informativeness differently from a human

expert, and offers no complete oracle for specification quality. Reported gains should therefore be read as evidence under the chosen metrics, rather than as a complete characterization of specification quality. Indeed, how to formally evaluate the quality of a specification beyond mere verifier acceptance remains an open problem, as recent efforts to benchmark and assess LLM-generated formal specifications make clear [37], [42].

Internal validity: LLM nondeterminism and Pass@k. LLM completions are stochastic. We fix the sampling temperature at 0.7 throughout, with OpenAI's `reasoning_effort=low` on the reasoning-tier models (`gpt-5.4-mini`, `gpt-5`); the sole exception is the industrial case study in Appendix J-E, which uses `reasoning_effort=high`. Because single-shot results vary across runs, we report Pass@1, Pass@3, and Pass@5 separately for the goal-dependent benchmarks (Table VIII); the goal-independent experiment in Section VI reports Pass@1 only, as its per-attempt cost is substantially higher. The Pass@1→Pass@5 gap recovers some hard cases on **OOPSLA**, **Frama-C**, and **list-S**, but is already marginal by Pass@3 on **NLA**, **numer-S**, and **pIp**. These rates describe the studied sampling configuration and may vary across independent runs.

Internal validity: data leakage. Bias and data leakage from the LLM's pretraining corpus are a concern, especially in program verification, where domain-specific data are scarce. Training-based methods and benchmark-specific retrieval pipelines are most vulnerable, since benchmark-adjacent corpora, curated examples, or synthetically generated programs may implicitly encode benchmark-specific patterns. SESPEC mitigates this by avoiding benchmark-specific training and dataset-dependent demonstrations: it does not fine-tune the model or train auxiliary components on the evaluation benchmarks, and its fixed prompts draw runtime content from the target program's symbolic facts and static features rather than from labels or expected specifications. Pretraining exposure to benchmark-related material remains possible. The reported gains provide evidence for the effectiveness of symbolic-execution-guided prompting under these experimental settings.

External validity: benchmark and baseline selection. The evaluation covers small and medium-size verification tasks from the AUTOSPEC suite, LIPuS, and CLAUSE2INV, plus our industry-derived **pIp** set and 400-case **SESPEC-400**. The **SV-COMP** subset is the AUTOSPEC-curated 21-program slice. These results characterize self-contained programs with the studied control and data structures; large industrial codebases and extensive external-library use remain outside the evaluated scope. Scaling functional verification further is constrained by the underlying proof problem. The first wall is the reach of automated theorem provers (ATPs), which time out or return *Unknown* on the nonlinear, quantified, and recursively defined obligations that realistic code induces, as our own Frama-C/WP runs repeatedly show (Appendix K). Migrating these obligations to an interactive theorem prover (ITP) changes this boundary rather than removing it: an ITP offers a richer proof surface for obligations that ATP backends fail to discharge, but it also shifts the burden to proof construction and proof search. Recent LLM-driven ITP efforts therefore remain evaluated on relatively small programs, while also suggesting that this route may become more scalable as LLM proof-script generation and proof search improve [37], [38], [42], [43].

External validity: memory model and verifier soundness. The verifier-backend completeness limits are discussed in Appendix K; here we isolate the soundness assumptions behind the reported verification results. We use Frama-C/WP's *Typed* memory model, which is known to be unsound for several C idioms, including pointer-to-pointer casts that violate effective types, accesses that reinterpret memory between different scalar types, reads or writes through unions that mix the active variant, and overlapping or misaligned accesses to the same region. The evaluated benchmarks use none of them: **SyGuS**, **OOPSLA**, **NLA**, **numer-S**, and **SV-COMP** are numerical loops without unions or type punning, while **Frama-C**, **pIp**, and **list-S** use structs, pointers, and singly linked lists but do not cast pointers across incompatible types or access overlapping union variants. Dynamic allocation through `malloc` is outside the modeling scope. Programs outside these memory-model assumptions are outside the soundness envelope of this evaluation.